A FIELD GUIDE FOR NOZ

Google Cloud TAM 技術面接対策本

問いから、判断へ。

クラウドの基礎を理解する。証拠から仮説を更新する。
顧客の業務へつなぎ、自分の言葉で説明する。

2026.10.31 / 図解追加版　30章 · 12ケース · 14図 · 約81千字　一次RRKを中心に

目次

基礎を学び、証拠から判断し、自分の言葉で伝える。

01　この本の使い方と、今回の準備範囲

02　システム全体を、一つの要求から見る

03　曖昧な相談から、判断と検証へ進む

04　TAMとして、技術判断を顧客の行動へつなぐ

05　Networking：一回の通信を分解して考える

06　Compute：実行方式・状態・Scaleを選ぶ

07　Storage・DB：保存、整合性、復旧を分ける

08　IAM・Security：主体、権限、境界を設計する

09　観測から事実を作る：Metrics・Logs・Traces

10　性能を考える：遅延・負荷・待ち行列と安全な緩和

11　信頼性とSRE：SLO・Error budget・HA・DR・改善

12　構成と移行の判断：顧客価値・制約・トレードオフ

13　nozの経験を、技術理解につなぎ直す

14　面接で伝える技術｜詰まった時の立て直しと回答の組み方

15　面接で伝える技術｜20場面の言い回しと、その後に話すこと

16　ケース01｜「システムが遅い」を、業務と測定へつなぐ

17　ケース02｜一部の利用者だけ接続できない

18　ケース03｜リリース直後に5xxが増えた

19　ケース04｜サービスからだけアクセスが拒否される

20　ケース05｜キャンペーンで負荷が急増した

21　ケース06｜更新直後の表示が古く、検索も遅い

22　ケース07｜一つのZoneが使えなくなった

23　ケース08｜バックアップは成功しているが、復旧できるか

24　ケース09｜月の途中でクラウド費用が急増した

25　ケース10｜短期間でGoogle Cloudへ移行したい

26　ケース11｜不審なアクセスと費用増加が見つかった

27　ケース12｜アラートが多く、手作業が減らない

28　直前に戻る、判断の早見

29　11/10を見据えた、学習と復習の回し方

30　出典と作成・確認の範囲

入口　CHAPTER 01

この本の使い方と、今回の準備範囲

最短11/10の一次RRKに向け、全体像を先に持ち、技術の仕組みを学び、条件の違うケースで判断を再現する。

nozのために、何をつなぐ本か

この本は、Oracle TAMとしての経験を出発点に、クラウドの原理、Google Cloudでの実現、顧客への説明をつなぐ学習用教材です。製品名を知ることから一歩進み、「どの情報があれば判断できるか」「なぜこの案を選ぶか」「何を確かめて成功とするか」を、自分の言葉で説明することを目指します。

nozがすでに行ってきた、顧客の状況整理、SupportやEngineeringとの連携、費用や作業影響の説明は、技術判断を支える経験です。そこへ、リクエストが通る経路、リソースが詰まる仕組み、データを守る設計、観測の限界を結び付けていきます。経験がある領域も、実装したことがあるとは仮定しません。

> **到達目標**
>
> 未知のケースで、目的・影響・制約を押さえ、重要な質問を選び、証拠で仮説を更新し、対策の効果と副作用を説明できる。答えを知っているケースだけでなく、知らない仕様が残るケースでも、確認先と次の安全な行動を示せる。

面接の案内と、本書の提案を分ける

区分	今回の準備への反映
採用担当者の案内として記録された内容	一次RRKは45分。日本語・5〜6問程度・過去経験と仮想状況の混在は、2026/10/29の口頭説明に基づく。技術知識と業界理解、顧客の立場に立った危機対応、関係者調整が説明された。
メールによる更新として元チャットに記録された内容	一次RRK 45分、二次RRK 2 — Case Study 60分、GCA 45分、G&L 45分。二次の提出物・作図・コーディング・言語は未確認。本書は一次を優先する。
nozの今回の希望	PCとスマホで読む。面接は最短11/10を想定。基礎も省略せず、理解済みの部分は読み飛ばせる。先に教材を持ってからケース練習を進める。
本書の学習上の提案	章ごとの自己説明、90〜150秒程度のケース回答例、復習順序、学習状態の記録。Googleの採点基準・合格条件・公式回答時間ではない。

面接形式の記述は元チャットとローカルの準備記録を参照しています。採用メール・録音原本を今回再照合したものではありません。日程確定や案内更新があれば、担当者の最新情報を優先します。コーディング不要という役割説明を、選考全体に実技がないという保証には広げません。

三つの読み方

1. **全体をつかむ：**「全体マップ」「共通の判断手順」「直前の早見」を読み、分からない用語に印を付けます。
2. **理由を学ぶ：**分野別の章を読み、仕組みと選択条件を理解します。既知の説明は飛ばし、自己確認に答えてから次へ進みます。
3. **使って確かめる：**ケースの解説を閉じ、自分の回答を先に話します。解説との差を一つ修正し、追加条件にもう一度答えます。

「読んだ」と「自力で使えた」は異なる状態です。各章の記録は自己申告であり、自動採点や合否判定は行いません。未着手も低評価ではありません。PCとスマホの記録は端末ごとに保存され、必要なら学習記録を書き出して移します。本文を開くのに外部スクリプトや外部フォントは不要です。

基礎が分からないときの戻り方

例えば「TLSで失敗している」と言われて説明できなければ、Networking章でDNS、TCP、TLS、HTTPの順序へ戻ります。「p99が悪化した」が曖昧なら、観測章で分布と母集団へ戻ります。ケースを解けない理由を、知識不足、観測不足、判断条件の不足、説明の不足に分けると、次に読む場所が決まります。

> **出典の読み方**
>
> 技術の仕様はGoogle Cloudなどの一次資料を参照し、2026/10/31時点で確認しました。学習用ケース、数値、会話、回答例は創作です。料金、制限値、機能提供地域、デフォルト、SLAの適用条件は変化し得るため、実務で使う際は対象構成の最新資料を確認します。本書はGoogleの公式面接教材ではありません。

システム全体を、一つの要求から見る

利用者からデータまでの経路に、制御・観測・業務目標を重ねる。製品の一覧を、仕組みの地図に変える。

利用者の操作が完了するまで

図 01　**利用者の操作を、一つの経路で追う**



一般的なWebアプリの概念図です。DNS・接続・TLSは要求を送るための準備であり、DNSサーバーがすべての業務データを中継する意味ではありません。応答は逆方向に利用者へ戻ります。

この図は学習用の典型例です。実際にはキャッシュや認証サービス、複数の内部サービス、メッセージキューが入ります。一つの画面操作が何本の要求を出すか、どこで並列になりどこで待つか、どの依存先の失敗が利用者へ伝わるかを確認します。サーバーの起動だけでは、利用者が注文を完了できるとは限りません。

DNSは名前から接続先を得る段階、TCPは通信相手への接続を確立する段階、TLSは通信を保護し相手を検証する段階、HTTPは要求と応答のやり取りです。HTTP/3など異なる通信方式もあるため、この順序は一般的なHTTPS/TCPの例として使います。「つながらない」を段階に分ければ、ログを探す場所も変わります。

三枚の地図を重ねる

地図	見たいこと	質問例
データの流れ	要求がどこを通り、どこへ保存されるか	遅いのは画面全体か、検索APIか、DB応答か。
制御と責任	誰が設定し、誰が実行を許可し、誰が復旧するか	アプリのサービスアカウントは何で、どのリソースに何の権限があるか。
観測	どの地点で何を測り、見えていない区間は何か	利用者の待ち時間と、LBの応答時間は同じ区間を測っているか。

管理用APIが失敗しても、既存の業務通信が続くことがあります。逆に、コンソールでリソースが正常に見えても、データを読むアプリは失敗することがあります。構成を管理する制御の経路と、実際の要求・データが流れる経路を分けて確認します。

要件を、設計の条件へ変える

「止めたくない」は出発点です。どの業務を、どの時間帯に、どの利用者へ提供するか、停止とデータ損失をどれだけ許容するか、費用や運用体制の制約を確認します。全機能を同じ水準にする必要があるとは限りません。受注の書き込みを優先し、集計画面を後回しにする選択も、業務との合意があれば成立します。

業界理解は、業界名を知ることだけではありません。公共分野ならデータ所在や契約・監査上の要求が設計条件になることがあり、小売なら繁忙期と売上への影響が容量計画に結び付きます。ただし、すべての公共顧客に同じ条件を当てはめず、対象の契約、データ、運用ルールを確認します。

Google CloudのWell-Architected Frameworkは、運用、Security、信頼性、費用、性能、持続可能性という観点で設計・運用を検討します。本書では今回の準備に直結する観点を中心に扱います。 公式: Well-Architected Framework

OCIの経験を使う手順

1. **原理を言う：**「一台への依存を減らし、健全な処理先へ要求を送る」。
2. **知っている実装に結ぶ：**OCIで経験したLBやComputeの構成を思い出す。
3. **Google Cloudの条件を確かめる：**LB種別、対象、健全性確認、スコープ、フェイルオーバー、課金、責任分界を確認する。
4. **同じ部分と違う部分を説明する：**名前の対応だけで仕様まで同じとは言わない。

解説 | 自己確認：なぜCPU使用率だけで「システムは正常」と言えないか

CPUが待っている間にDBロック、ディスクI/O、外部API、接続プール、ネットワークなどで要求が滞留している可能性があります。CPU平均が低くても一部のインスタンスや時間帯だけが飽和していることもあります。利用者の成功率と応答時間から、経路に沿って証拠を集めます。

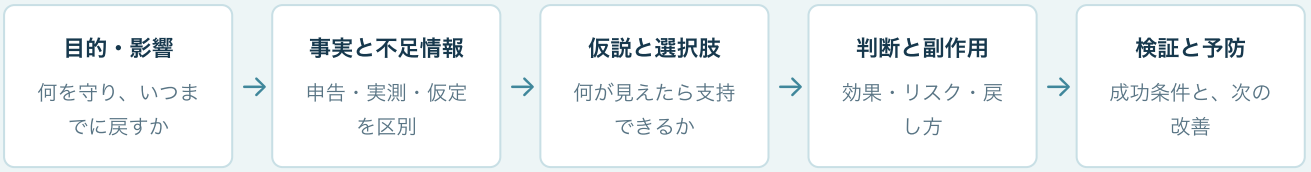
考え方　CHAPTER 03

曖昧な相談から、判断と検証へ進む

質問を大量に並べるより、次の判断を変える情報を選ぶ。影響緩和・調査・連携は必要に応じて並行する。

回答の骨組み

図 02　回答を、判断の順番で組み立てる



学習用の骨組みです。すでに分かっている情報は繰り返し質問せず、判断を変える不足情報を選びます。用語の説明に、この長い型を毎回当てはめる必要はありません。

これは本書の学習用の型です。用語説明にもすべての段階を強制するものではありません。障害では「重大な影響があるので関係者を招集し、安全な緩和策を検討しながら、原因の切り分けを進める」のように並行して動きます。設計相談では、目的と制約を先に合意し、選択肢の比較に時間を使います。

一問に、判断上の目的を持たせる

質問	何が変わるか
どの業務のどの操作に、どの程度の支障があるか。	緊急度、対応体制、優先して守る機能。
普段と現在の所要時間、成功・失敗、発生時刻は。	症状の定義、調べる時間帯、成功要求と失敗要求の切り分け。
影響する利用者・地域・機能と、影響しないものは。	共有依存先か局所要因か、正常系との比較方法。
直前に変更、負荷、データ量などの変化はあるか。	優先仮説、比較対象、戻せる変更の候補。
どの状態まで、いつまでに回復すれば業務を続けられるか。	暫定対処の成功条件、機能縮退や迂回の許容範囲。

最初から全質問を読み上げる必要はありません。まず業務・影響を押さえ、既に与えられた情報を使って、次に判断が変わる一問を選びます。面接官から情報が得られなければ、「ここでは単一リージョンのWebサービスと仮定し、違えば方針を修正します」と、影響する仮定を明示します。

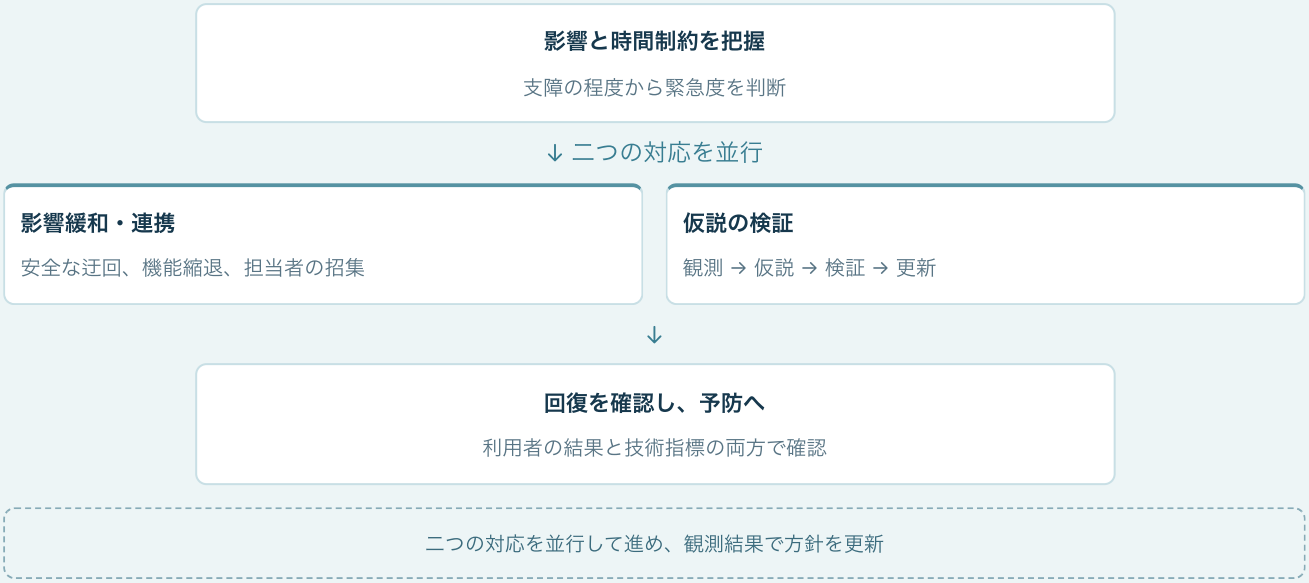
情報を四つに分ける

確認済み：対象時刻と測定条件が分かるログやメトリック、再現結果。**申告：**「普段2秒、今20秒」などの聞き取り。**仮説：**DB待ちが伸びた可能性。**未確認：**対象要求のトレース、データ量、直近の変更。この区別は、情報を信用しないためではなく、次に何を確かめるかを見えるようにするためです。

観測した後に推論を書く順序を守ります。「CPUが高い」から「CPU不足が根本原因」へ飛ばず、負荷増、処理量増、実行計画変化、再試行増などの説明が成立するかを確認します。逆に、原因未確定でも、十分な根拠と安全性の確認がある暫定策は検討できます。

仮説は、予測と検証を一組にする

図 03　**重大な影響では、緩和と調査を並行する**



原因が確定するまで緩和を待つ手順ではありません。安全に実行できる案を関係者と確認し、変更の副作用と証拠保全にも配慮します。業務を継続できていても重大な支障はあり得ます。

> 学習例：「外部APIが遅くなり、アプリの待ち時間が増えた」と仮定するなら、対象要求のトレースで外部呼び出し区間が増えている、外部APIを使わない操作は比較的正常、という予測を置きます。観測が合わなければ、DBやアプリ内部の待ちへ仮説を移します。単に「ネットワークもDBも確認する」より、調査の目的が伝わります。

Google SREのトラブルシューティング解説も、システムの理解と観測を基に仮説を立て、支持・反証する証拠で更新する進め方を説明しています。重大な影響では、原因確定前の緩和と証拠保全も扱います。 公式: Effective Troubleshooting

対策を説明する五つの要素

1. **狙う効果：**どの仮説に対して、何の待ちや失敗を減らすか。
2. **成立条件：**容量、権限、整合性、戻し方など、実行に必要な条件。
3. **副作用：**停止、再実行、データ損失、負荷の移動、追加費用。
4. **実行と判断の担当：**誰が承認し、誰が操作し、誰が結果を確認するか。
5. **成功・中止条件：**業務が回復したと判断する指標と、悪化時に止める条件。

元の遅延ケースから覚える三つの違い

普段の状態：以前の検索は約2秒。**業務上必要な状態：**5秒以内なら通常の人数で受注をさばける。**現在の状態：**待ちが増えているが、まず測定区間と分布、失敗有無を確かめる。この例の2秒と5秒は学習用の条件であり、普段の2秒が自動的にSLOになるわけではありません。

当面は合意した業務条件の回復を目指し、平常との差は継続調査できます。ただし「5秒以内」が平均なのかp95なのか、対象操作と観測期間は何かを確認します。業務の回復、技術指標の回復、根本原因の確定は、それぞれ別に判定します。

解説 | 自己確認：変更後に改善したら、変更が原因を証明したと言えるか

単独の変更と再現性があり、他の変化を排除できるほど根拠は強まりますが、同時に負荷や複数リソースが変われば寄与を分離できません。「変更後に改善した」という観測と、「原因を特定した」という結論を分けます。復旧後の追加分析や条件をそろえた検証を計画します。

TAMとして、技術判断を顧客の行動へつなぐ

顧客の懸念を、調査可能な論点と判断条件に変える。連携を依頼する内容と、noz自身の判断を具体化する。

「専門チームに渡す」で終わらない

TAMがすべてを実装する必要はありません。一方、「Supportへ連絡します」だけでは、何を解決したいかが伝わりません。顧客影響、発生時刻、影響範囲、構成、直近の変更、集めた証拠、確認したい仮説、求める判断、時間制約をそろえて依頼します。未知の製品仕様を推測で補わず、確認する担当と、それまで進められる行動を示します。

> 学習用の依頼例：「受注検索のp95が業務上の許容値を超え、電話待ちが増えています。アプリの処理区間ではDB接続待ちの増加を観測しました。接続上限・プール設定とDB側の飽和を確認したく、アプリ担当とDB担当に並行調査をお願いします。アプリ増設はDB接続を増やす恐れがあるため、実施条件を先に確認したいです。」

危機対応の役割を分ける

大きな障害では、対応全体の統括、技術作業、顧客・経営層への連絡を分けると、調査と説明の両方を進めやすくなります。TAMが必ず統括者になるとは限りません。既存のインシデント体制、権限、顧客との契約に合わせて、自分の役割と窓口を明確にします。

Google SREのIncident Responseでは、統括・連絡・運用の役割、作業記録、連携の構造を扱っています。本書の顧客説明例は、その考え方を学習用に具体化したものです。 公式: Incident Response

確定情報が増えていなくても、更新できること

- **分かっていること：**顧客影響、対応中の範囲、現在の回復状況。
- **変化と実行：**どの確認が終わり、何を並行して進めているか。
- **不明と次の判断：**未確定の原因、確認中の緩和策、顧客へ依頼する判断。
- **次の連絡：**合意した時刻または条件。復旧時刻が未定なら、確約と見込みを分ける。

技術担当の「問題なし」と顧客の「まだ遅い」が食い違ったら、どちらかを否定する前に測定対象をそろえます。インフラが正常でも、利用者経路や特定機能が回復していないことがあります。顧客に必要な完了条件を、実測と業務確認の両方で確かめます。

意見が割れたときの比較

営業は早期の再開、運用は変更抑制、開発は原因分析を重視するかもしれません。まず共通の目的を「安全に受注を継続する」などの業務へ置き直し、案ごとに期待効果、失敗時の影響、可逆性、所要時間、追加費用を比較します。技術的に妥当な案でも、承認者や実行体制がなければ実行できません。決める人と検証する人までそろえます。

過去経験と仮想ケースの話し方

質問	説明すること
過去に何をしたか	状況、自分の責任、観測、自分の判断・行動、他者の行動、結果、学び。実際に行っていない確認は後から足さない。
この状況ならどうするか	確認すべき前提、仮定、優先仮説、暫定策、検証、残るリスク。未確定の仕様は確認先を示す。
なぜその案を選んだか	比較した選択肢と制約、採用理由、弱点、判断を変える条件。

解説 | 自己確認：「自分は調整しただけ」と感じたら、何を具体化するか

何の情報が不足・矛盾していたか、どのリスクを優先したか、誰に何の判断を依頼したか、顧客が何を決められるようになったかを説明します。実装を自分の成果へ置き換える必要はありません。調整による時間短縮や損失回避は、測っていなければ数値化しません。

Networking：一回の通信を分解して考える

DNS、TCP、TLS、HTTPから経路・Firewall・LB・Private接続まで、到達性と業務成功を分けて判断する。

この章の到達点と早見表

「つながらない」を一つの故障名として扱わず、名前解決、接続、暗号化、アプリケーション応答のどこまで進んだかを説明できるようにします。Networkは経路だけの話ではありません。通信元・通信先・名前・プロトコル・ポート・時刻をそろえると、関係者が同じ現象を調べられます。

- DNSで名前を解決できても、宛先への接続や業務処理の成功は未確認です。
- 経路は「どこへ送るか」、Firewallは「通してよいか」、IAMは「誰が何をしてよいか」を扱います。
- Load Balancerの正常判定と、利用者の業務成功を別々に検証します。
- Timeoutは失敗を観測する期限です。処理が取り消されたか、後で完了したかは別に確かめます。

1. 利用者の一回の操作を通信の流れに分解する

図 04　**通信は、どこまで成功したかで分ける**



TCPを使うHTTPSの調査観点を単純化した図です。矢印は次に確かめる段階を示し、前段の成功は後段の成功を保証しません。HTTP/3など実際の方式や構成に合わせて読み替えます。

利用者 → DNSで名前を解決 → 宛先へ接続 → TLSで接続を保護 → HTTP要求 → Load Balancer → アプリケーション → DB・外部サービス → 応答

これはHTTPSを使う一般的なWebアプリの学習用の図です。実際にはProxy、CDN、VPN、認証基盤などが加わり、HTTP/3ではTCPの代わりにQUICを使います。図を事実と決めつけず、最初に実際の構成と測定区間を確認します。

DNS：名前から接続先の情報を得る

DNS（Domain Name System）は名前に対応するIPアドレスなどの情報を問い合わせる仕組みです。利用者は通常、手元や社内のResolverに問い合わせ、必要に応じて権威DNSから情報を得ます。結果はキャッシュされるため、設定を変更しても全員が同時に同じ値を見るとは限りません。TTLはキャッシュの有効期間に関する値で、変更が全世界へ一斉に配布される時刻を保証する値ではありません。

障害時には「名前が引けるか」だけでなく、どのResolverから、どの名前を、いつ引き、どの結果を得たかを確かめます。社内だけ失敗するなら社内DNSやPrivate DNSの可視範囲が候補になり、特定端末だけなら端末側のキャッシュや接続先設定も候補になります。名前解決の失敗と、正常に得たIPが意図と違うことを分けます。

TCP：相手と接続を確立してデータを運ぶ

TCP（Transmission Control Protocol）は、順序や再送などを管理する通信方式です。接続開始には相手から応答を受ける必要があり、ルート、Firewall、NAT、相手の待受状態などが影響します。接続Timeoutなら「返答が観測できなかった」という状況を表します。どの機器が破棄したかを、その文字列だけで断定できません。Connection refusedなら、その経路上の何かが拒否を返した可能性を考え、待受ポートやProxyも確認します。

IPアドレスへのpingが通ることは、目的のTCPポートやアプリケーションが利用できる証明ではありません。ICMPを許可していない正常なサービスもあります。業務で使うプロトコルと宛先で確認することが必要です。

TLS：通信相手の確認と暗号化

TLS（Transport Layer Security）は、証明書などを使って相手を確認し、通信を保護します。接続はできるがTLSで失敗する場合、証明書の期限、ホスト名との一致、証明書チェーン、信頼する認証局、クライアント時刻、対応するプロトコルなどを確認します。暗号化されているだけで、アプリの利用権限まで与えられるわけではありません。

Load BalancerでTLSを終端する構成では、利用者とLB、LBとBackendは別の接続です。利用者側で証明書が正常でも、Backend側の暗号化や認証は別の確認対象になります。検証のために証明書検証を無効化すると、何が失敗しているかの切り分けに使えても、安全な恒久対策にはなりません。

HTTP：業務要求と応答を扱う

HTTPはメソッド、URL、ヘッダー、本文、ステータスコードなどで要求と応答を表します。4xxは要求や認証・認可などを調べる手掛かり、5xxはサーバー側や上流依存の異常を調べる手掛かりです。ただし、どの層がコードを生成したかが重要です。403を見て必ずIAMだと判断したり、502を見て必ずDB障害だと判断したりしません。

解説 | もう一段理解する：遅延を測る区間

利用者の体感時間には、DNS、接続確立、TLS、サーバー処理、応答転送、画面描画などが含まれます。Backendの処理時間が短くても、接続確立や外部API待ち、クライアント描画で遅くなる場合があります。「サーバー側の平均応答時間」と「ユーザーの検索開始から結果表示まで」をそのまま比較すると、測定対象の違いを障害と誤認す

るおそれがあります。開始点・終了点・集計方法・成功要求と失敗要求の扱いをそろえて比較します。

2. 経路・Firewall・Load Balancerの役割を分ける

Route（経路）は宛先に対して次にどこへ送るかを決めます。Firewallは通信の方向、送信元、宛先、プロトコルなどの条件で許可・拒否を制御します。行きの経路だけでなく戻りの経路、NATで変わる送信元、LBやProxyが作り直す接続を含めて考えます。「同じVPCだから通る」「Firewallを一つ許可したから通る」という説明は確認が不足しています。

Load Balancerは複数Backendへの要求分散を担います。L4は主にIPやポートなど、L7はHTTPのHostやPathなどを使う判断に向きます。Health checkは特定の観測点・条件に対する検査です。簡単な生存確認だけなら、DB接続が失敗しているBackendを正常と判定する場合があります。逆に、一時的な重さで全台が不健康と判定されると、利用可能な経路を減らす可能性もあります。

3. Google Cloudでの実現例

Google CloudのVPC networkはグローバルなリソースで、Subnetはリージョンに属します。この単位を押さえたうえで、構成図にはプロジェクト、VPC、Subnet、リージョン、接続点を書きます。共有や接続による実効範囲は、名前が似ているサービスから推定せず設定で確認します。公式：VPC networks

Cloud DNSにはPublic zoneとPrivate zoneがあり、公開先・可視範囲が異なります。Google Cloud内で名前が引けることと、社内やインターネットから同じ名前が引けることは同義ではありません。公式：Cloud DNS overview

Cloud Load Balancingでは、HTTP(S)のL7処理にApplication Load Balancer、TCPなどのL4処理にNetwork Load Balancerを検討します。External／Internal、Global／Regional、Proxy／Passthroughなどの条件があり、送信元IP、対応プロトコル、Backendの範囲を要件に照らして選びます。名称を一つ挙げるだけで選定を完了させません。公式：Cloud Load Balancing overview

4. Private接続は目的を具体化して選ぶ

「Privateにしたい」と言われたら、公開IPを持たせたくないのか、インターネット経由を避けたいのか、特定のサービスだけ公開したいのか、社内から閉域接続したいのかを確認します。Privateという単語だけでは、認証、暗号化、権限管理、データ持出し防止の全要件を満たすか判断できません。

接続方式を比較するときは、到達先、通信方向、名前解決、アドレス重複、経路交換、暗号化、冗長性、帯域・遅延、障害時の切替を確認します。APIへのPrivateなアクセスと、ネットワーク全体の接続は要件が異なります。単に「接続サービスを入れる」ではなく、必要な通信が成立する条件と、不要な通信を防ぐ条件を分けて設計します。

用途から選ぶGoogle Cloudの接続方式

まず「ネットワーク同士をつなぐ」のか、「Google APIや提供サービスへアクセスする」のかを分けます。前者の接続を作っても、後者に必要なDNS・経路・アクセス設定まで自動で完成するとは限りません。次は代表的な入口であり、相互に一対一で置き換える製品一覧ではありません。

方式	つなぐもの・主な目的	選択前に確認する条件
Cloud VPN	オンプレミスなどのNetworkとVPCを、IPsec Tunnelで接続する。	相手側VPN機器、必要な帯域、経路、冗長化、暗号化要件。HA VPN over Cloud Interconnectという組合せもある。
Cloud Interconnect	Dedicated／Partnerなどの方式で、外部NetworkとGoogle Cloudの接続を用意する。	接続拠点・事業者の手配、必要な容量、冗長経路、納期、費用。接続方式だけで暗号化の要件を満たしたと判断しない。
Private Google Access	External IPを持たないGoogle CloudのVMなどから、対応するGoogle API・Serviceへアクセスする。	Subnetの設定、対象API、DNSと経路。VPC内に専用のPrivate endpointを作る方式とは異なる。オンプレミス向けは別の設定・経路条件を確認する。
Private Service Connect	代表例はVPC内のEndpointを通じた、Google APIや別VPCから公開された対応Serviceへのアクセス。	対象Serviceが対応する接続方式、Endpoint・DNS・利用者側設定。Network全体を自由に接続する方式とは扱わない。
private services access	VPC Network Peeringを使い、対応するGoogle・第三者のManaged service側Networkへ接続する。	Serviceの対応、接続の作成、IP範囲と経路の計画。Private Service ConnectやPrivate Google Accessと名前だけで同等と判断しない。

Cloud VPN／Interconnectの選び方は公式：Choosing a Network Connectivity product、ServiceごとのPrivateアクセスは公式：Private access options for servicesで確認できます。面接では最初に接続元・接続先・通信方向を述べ、次に必要な方式を絞ると説明が明確です。

5. TimeoutとRetryを安全に説明する

Timeoutは、処理をどれだけ待つかを決める制御です。短すぎると正常な処理を失敗として扱い、長すぎると待機中の接続やメモリを占有します。利用者の許容時間から、LB、アプリ、DB、外部APIなどの時間配分を考えます。内側の処理が外側の期限を超えて続くと、利用者は失敗を見ているのに処理だけ残ることがあります。

Retry（再試行）は一時的な失敗に有効な場合がありますが、再試行が重なると負荷を増やします。回数・全体期限を制限し、待ち時間を増やすBackoffと、再試行時刻を散らすJitterを検討します。更新要求では、応答を受け取れなかっただけで処理済みの場合があるため、二重登録を防ぐ冪等性が必要です。冪等性は同じ要求を繰り返しても意図した最終結果が変わらない性質です。Cloud Storageでも操作や前提条件により再試行の安全性が異なります。公式：Retry strategy

6. OCI経験から考えるときの橋渡し

OCIで使ってきたSubnet、Route、Firewall、LB、DNSという役割の分解は、Google Cloudを理解する出発点になります。ただし、リソースの範囲、ルールの適用先、優先順位、LBの動作、Privateアクセス方式をそのまま移し替えません。面接では「役割は共通ですが、Google Cloudではこの条件を確認します」と説明できると、経験と新しい環境の両方を扱えます。

7. 小さなケース：社内からだけアクセスできない

> 学習用の創作例です。公開Webシステムに、自宅回線からは接続できますが、社内ネットワークからだけ接続できません。業務担当者は「クラウドのFirewallが原因」と考えています。

1. 同じURL・時刻・操作で比較し、影響する拠点と利用者、業務継続の可否を確認します。
2. 社内と社外でDNS結果を比較します。異なれば意図したSplit DNSか、古い設定かを確かめます。
3. 名前解決が同じなら、接続、TLS、HTTPのどこで失敗するかを分けます。社内Proxyや送信元IP制限も候補です。
4. クラウド側の受信記録、LBの応答、社内側の観測を時刻で対応させます。記録がないことだけで、クラウド手前で落ちたと断定しません。収集範囲やSamplingも確認します。
5. 原因候補を絞って必要最小限の修正を検証し、他の接続元への影響とRollbackを用意します。

8. よくある落とし穴

- 「一部から成功する」ことを、ネットワーク全体が正常な証拠にする。
- 証明書エラーを避けるため、検証無効化を恒久対策として提案する。
- 接続Timeoutを理由に、全通信を許可するFirewall変更を先に行う。
- Timeoutを延ばしただけで処理能力が増えたと考える。
- 「Privateだから安全」とし、IAMや監査を確認しない。

9. 面接での短い説明例

> 「まず、影響する業務と接続元の範囲、発生時刻を確認します。そのうえで同じ要求がDNS、接続、TLS、HTTPのどこまで進むかを比較し、経路・許可設定・Proxy・Backendの候補を絞ります。安全な緩和を並行して検討し、修正後は接続成功だけでなく対象業務が必要な時間内に完了することまで確認します。」

10. 自己確認

1. DNSが成功しpingも通るのに、Webサイトが使えない理由を三つ説明してください。
2. Timeoutを延ばす案とRetryを増やす案には、どのような副作用がありますか。
3. Internal Load Balancerを使えばIAMやTLSは不要と言われたら、どう説明しますか。

解説 | 確認の観点を開く

1：目的ポートの遮断、TLSの検証失敗、アプリの認可・処理失敗などが残ります。2：待機リソース増加、復旧の遅延、再試行の集中、更新の重複が候補です。3：到達可能な範囲と、利用主体の権限、通信保護は別の制御です。要件に応じて組み合わせ、実際の接続点と責任分界を確かめます。

Compute：実行方式・状態・Scaleを選ぶ

VM・Container・Serverlessの責任分界を理解し、状態と下流容量を含めてScaleを判断する。

この章の到達点と早見表

VM、Container、Serverlessを「新しい方がよい」という順序で扱わず、必要な制御と運用負荷、アプリケーションの性質から選びます。性能問題では、CPUだけでなく、メモリ、I/O、接続、待ち行列、下流サービスまで含めて容量を考えます。

- VMはOSを含む実行環境の制御を得る一方、その管理が必要です。
- Containerはアプリと依存関係をまとめる単位です。Container化だけで可用性や伸縮性が完成するわけではありません。
- Serverlessでもサーバーは存在します。管理の一部をサービスへ委ね、実行方式の条件に合わせます。
- 台数を増やす前に、状態の置き場所、起動時間、下流の上限を確認します。

1. CPU・メモリ・I/Oの関係を押さえる

CPUは計算を実行し、メモリは処理中のデータを保持し、I/Oはディスクや Networkなどとのデータのやり取りを担います。処理が遅くても、CPUが使い切られているとは限りません。DB応答待ち、Lock待ち、接続Pool待ちでは、CPU使用率が低いまま待ち時間が増えることがあります。

メモリ不足では、OSや実行環境の挙動に応じてSwap、Garbage Collection、プロセス終了などが起こり得ます。使用率だけでなく増え方、失敗時刻、再起動、要求あたりの消費量を確認します。「CPUが平均30%だから余裕がある」とは言い切れません。一つのCPU Coreや一部のInstanceだけが詰まっている場合、平均では見えにくくなります。

2. VM・Container・Serverlessを役割で理解する

VM：OSを含めた実行環境を選ぶ

VM（Virtual Machine）は仮想化された計算機です。Guest OS、Middleware、アプリの構成を広く制御できるため、既存ソフトウェアの要件やOS設定への依存を扱いやすくなります。その分、Guest OSのPatch、監視、容量、バックアップ、再構築手順などの担当を決める必要があります。

Compute EngineはGoogle CloudのIaaSとしてVMなどを提供します。まずワークロードのCPU・メモリ・I/O要件、対応OS、配置先、可用性、ライセンスと運用体制を確認して候補を選びます。高性能なMachine typeを選ぶだけで、アプリやDBの待ち時間まで自動的に改善するわけではありません。公式：Compute Engine overview

Container：アプリの実行物をまとめる

Containerはアプリと依存ライブラリなどをImageにまとめ、隔離された実行環境で動かす方式です。一般的なLinux ContainerはHostのKernelを共有し、VMと同じ境界ではありません。Imageは実行の元になる成果物、Containerはその実行中のInstanceと考えると整理できます。

同じImageを使うことで環境差を減らせますが、外部DB、Secret、ネットワーク、CPU種別、実行権限、永続Volumeなどの条件は残ります。「手元でContainerが動く」と「本番環境で要件を満たす」の間には、実行基盤との適合確認があります。

Kubernetes：望ましい状態を維持する仕組み

Kubernetesでは、Podが一つ以上のContainerをまとめる実行単位になり、NodeがPodを動かす計算資源になります。Controllerは「必要な数のPodを動かす」などの望ましい状態と現状を比較して調整します。Serviceなどの仕組みにより、入れ替わるPodへ到達するための接続点を用意します。

GKEはマネージドなKubernetes実装です。StandardとAutopilotなどの運用方式により、Googleと利用者が担当する範囲が変わります。どちらでも、アプリの正しさ、要求する資源、Identity、データ、サービス要件を利用者が設計する必要があります。「マネージドなので運用不要」ではなく、何の運用が減り、何が残るかを述べます。公式：GKE overview

Serverless：実行基盤管理を委ねる

Serverlessは、利用者が個々のサーバーを常時管理する負担を減らす実行モデルです。HTTP要求、Event、Jobなど、どの形で起動・終了するかを確認します。短い要求処理、長い処理、常駐処理は同じ前提では扱えません。対応する実行方式、ライフサイクル、外部接続、状態保持、起動時の遅延を要件と照合します。

Cloud Runでは、実行方式に応じたContainer runtime contractに合わせます。例えばServiceでは要求を受け付ける待受設定が必要で、標準の書込み可能なContainerのファイルシステムは永続データの保存先として扱いません。データの保存先と、再起動・置換後にも残す情報を明示します。公式：Container runtime contract

3. Scale upとScale outを使い分ける

Scale upは一台の資源を増やし、Scale outは処理する台数を増やします。Scale upは既存構成の変更が少ない場合がありますが、上限、再起動、費用などを確認します。Scale outは負荷を分散できる場合に有効ですが、処理を分担できない設計や共有DBのボトルネックは、そのまま残ります。

Statelessは、ある要求の処理に必要な継続的な状態を、そのInstanceだけに閉じ込めない設計です。ログインSessionを一台のメモリにだけ置くと、別Instanceへの振分けや再起動で利用者の状態を失う可能性があります。共有の保存先やTokenなどを検討しますが、外部化すると、その保存先の可用性、遅延、容量、Securityが新しい

設計対象になります。

解説 | 理解を深める：Statefulな処理を増やす難しさ

受注処理を複数台で実行しても、同じ在庫を更新するなら整合性を守る仕組みが必要です。Local cacheを持つ複数Instanceでは、更新通知や期限による無効化も考えます。StatefulだからScale outできないと一律には言えませんが、状態の分割、複製、競合制御、復旧時の所有権などを追加で設計します。TAMは「台数を増やせるか」を、アプリ担当とデータ担当の両方へ確認します。

4. Autoscalingを制御のループとして理解する

Autoscalingは、観測した負荷などに応じて資源を増減する仕組みです。観測、判断、Instanceの作成、起動、準備完了までに時間がかかります。突発的な負荷に必ず即応できるとは限りません。最小・最大数、Scaleの指標、Warm-up、Quota、実際の容量、下流サービスの耐容量を確認します。

Cloud Run Serviceでは、CPUや要求のConcurrencyなどを用いるAutoscalingと、Instance数の設定が関係します。Concurrencyは一つのInstanceが同時に処理する要求数に関する設定であり、利用者総数そのものではありません。処理がCPU中心かI/O待ち中心かにより、適切な設定は変わります。公式：About instance autoscaling 公式：Maximum concurrent requests

Concurrencyを高くすると、少ないInstanceに多くの要求を割り当てられる場合がありますが、メモリや内部Lockの競合が増える場合もあります。低くすると一Instanceの競合は減っても、必要なInstance数や起動の回数が増える場合があります。負荷試験で応答時間、Error、資源使用、Instance数、下流接続数を同時に観測して調整します。

5. 実行基盤を選ぶときの判断表

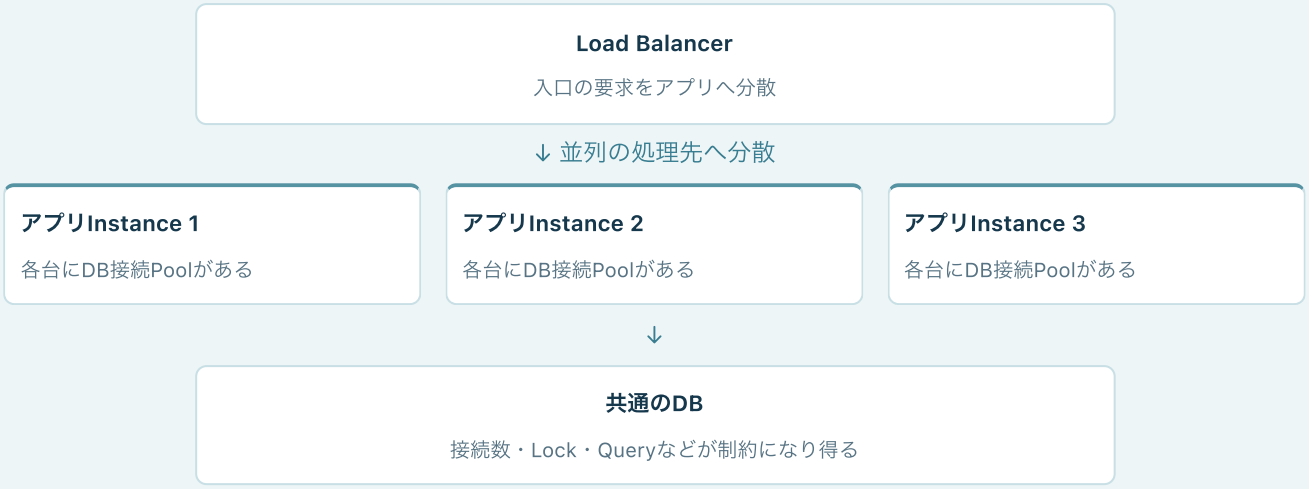
確認する条件	判断へのつなげ方
OSや特殊な常駐ソフトへの依存	VMで必要な制御を得る案を検討する。運用担当と再構築方法も決める。
KubernetesのAPIや運用資産が必要	GKEを候補にし、チームが管理する範囲と複雑さを評価する。
対応する実行方式でアプリを動かせる	Cloud Runを候補にし、状態・起動・接続・実行制約を確認する。
突発的なピークに対応したい	Scaleするまでの時間、最低容量、Quota、下流上限を検証する。
復旧を人の手順から減らしたい	Image、構成管理、自動再作成、Health checkを組み合わせる。

6. OCI経験からの橋渡し

OCIでのVM、Container、負荷分散、Auto Scalingの経験から、「どの資源が飽和するか」「どこに状態があるか」「誰がPatchや復旧を担当するか」を考える習慣を使えます。サービス名の対応表は入口で、実行モデルや責任分界が同じである証拠にはなりません。面接では、自分が設計・実装した範囲と、SupportやEngineeringと調整した範囲を分けて話します。

7. 小さなケース：Webを増やしたのに遅い

図 05　**アプリを増やすと、下流への接続も増え得る**



3台は概念上の例です。矢印は要求の経路を示します。各台のPool上限と台数から接続の最大規模を考えますが、実際の接続数・待ち・拒否は別に観測します。アプリの増加だけでDBの処理能力は増えません。

> 学習用の創作例です。負荷増加に対してWeb Instanceを二倍にしましたが、遅延が改善せず、DBの接続待ちが増えています。

まず利用者の応答時間とErrorがどう変わったかを確かめます。仮説は「Webの計算資源ではなく、共有DBや接続Poolが制約になっている」です。InstanceごとのPool上限にInstance数を掛けると、DBへ向かう接続の総数が増える可能性があります。ただし、設定上の上限と実際の接続数は分けて確認します。

暫定策には、流入の制御、不要なRetryの抑制、重い処理の延期などがあります。恒久策は、Query、Index、接続管理、Cache、処理の非同期化、容量などを原因に合わせて比較します。WebのScale outを止める場合も、その影響を評価し、利用者側のErrorへ付け替えただけになっていないかを確認します。

8. よくある落とし穴

- Container化すればアプリが自動的にStatelessになると考える。
- 平均CPUだけを根拠に余裕や原因を判断する。
- Autoscalingを、Quotaや下流上限を超える仕組みだと説明する。
- Serverlessなら起動遅延や容量設計が不要と考える。
- VMの再起動成功を、業務復旧の完了とする。

9. 面接での短い説明例

「実行基盤は、必要なOS制御、アプリの実行方式、状態の持ち方、チームの運用能力から選びます。既存アプリの制御が必要ならVM、Kubernetesの機能や資産を活かすならGKE、対応する実行モデルで管理負荷を減らせるならCloud Runを比較します。Scaleのしやすさだけでなく、起動時間と下流の容量を含めて検証します。」

10. 自己確認

1. CPU使用率が低いのに応答が遅い理由を二つ説明してください。
2. SessionをLocalメモリに置くアプリをScale outすると、何を確認する必要がありますか。
3. Cloud RunのConcurrencyを上げる前に、何を測定しますか。

解説 | 確認の観点を開く

1：DB・外部API・Lock・接続Pool待ち、一部Coreだけの飽和などが候補です。2：振分け、Session喪失、再起動、外部化先の耐障害性や遅延を確認します。3：要求あたりのCPU・メモリ、並行処理の安全性、遅延分布、Error、下流接続数を測り、変更後の比較条件をそろえます。

Storage・DB：保存、整合性、復旧を分ける

Block・File・ObjectとDBの役割を理解し、Query・整合性・Replication・Backupから業務要件に合う判断を行う。

この章の到達点と早見表

データの保存方式は、「どのサービスを知っているか」より、アプリが必要とする読み書きの単位、整合性、遅延、復旧条件から選びます。DBの遅さでは、保存容量と処理性能を分け、複製とバックアップでは、守りたい障害の種類を分けます。

- Blockはディスクとして、Fileは共有ファイルとして、Objectは名前を持つデータとしてアクセスする方式です。
- DBの性能は、CPUやディスクだけでなくQuery、Index、Lock、接続数、データ分布でも変わります。
- Replicationは別の場所にも状態を持つ仕組み、Backupは過去の状態へ戻すための材料です。
- HAはサービス継続、DRは大きな障害からの回復を狙います。対象障害と復旧目標を具体化します。

1. Block・File・Objectはアクセス方法が違う

Block Storage：OSからディスクとして使う

Block StorageはデータをBlock単位で扱います。一般にはOSがその上にファイルシステムを作り、アプリやDBが利用します。容量だけでなく、IOPS（一定時間内に処理できるI/O回数）、Throughput（一定時間内の転送量）、Latency（I/O一回にかかる時間）を確認します。小さなRandom I/Oが多いDBと、大きなファイルの連続読込みでは、必要な性能が異なります。

Google CloudではHyperdiskやPersistent Diskなどが永続的なBlock Storageの候補です。対応するMachine seriesや性能の設定方法はDisk種別で異なるため、「容量を増やせば必ず性能が増える」と一括りにしません。Localな一時領域を使う場合は、停止や障害などで失う条件を確認します。公式：Choose a disk type

File Storage：ファイルとDirectoryを共有する

File Storageはファイル名、Directory、ファイル操作などの仕組みを提供します。複数の計算資源から同じファイルを使う既存アプリでは候補になります。ただし、同時更新のLockや権限、必要なプロトコル、Metadata操作の多さ、接続先の耐障害性を確認します。共有できることと、同時に安全に更新できることは別です。

FilestoreはGoogle CloudのマネージドなFile serverです。アプリのファイル操作とサービスの対応機能、性能、可用性、接続方式を照合して採用を判断します。公式：Filestore overview

Object Storage：ObjectをAPIなどで読み書きする

Object Storageは、内容とMetadataを持つObjectを名前で指定して読み書きします。画像、文書、配布物、Backupなどの保存に適します。Cloud StorageではBucketにObjectを保存します。DirectoryやMountの機能があっても、既存アプリが必要な更新・Lock・原子的な操作を同じように使えるかは確認が必要です。公式：Cloud Storage overview

Storage classは、アクセス頻度や保存期間などを費用に結び付ける判断対象です。容量単価だけでなく、読出し、操作、転送、最低保存期間に関する条件などを確認します。安い保存先に変えた結果、頻繁な取得や復旧時の費用が増える場合があります。料金は採用時点の公式条件で見積もります。

2. DBを選ぶ前に、データと処理を定義する

Relational DBは表と関係を使ってデータを表し、SQLやTransactionで関連する更新を扱います。NoSQLは単一の方式ではなく、Key-value、Document、Wide-columnなどの異なるデータモデルを含みます。「NoSQLなら速い」「SQLならScaleしない」という分類では、具体的なアクセスパターンを説明できません。

最初に、何をKeyに検索するか、どの項目を一緒に更新するか、どれくらいのデータ量と要求数があるか、最新値が必要か、分析と業務更新のどちらが中心かを確認します。短い業務Transactionを大量に扱う処理と、大量データを集計する処理では、最適な実行方法が異なります。

Google CloudのDB・分析サービスを選ぶ入口

次の表は、主要な候補を考え始めるための整理です。サービス間には機能の重なりがあり、更新か分析かだけで採用が決まるわけではありません。既存DBとの互換性、Transactionの範囲、データの置き方、実際の負荷と運用を確認します。Cloud SQLなどの位置付けは公式：Google Cloud databasesも参照できます。

候補	データモデル	主に考える用途	選択を左右する条件
Cloud SQL	MySQL・PostgreSQL・SQL ServerのRelational DB。	既存のSQL資産を活かす業務更新・Transaction。	Engine・拡張機能の互換性、必要な容量、接続数、HA・復元、運用要件を確認する。
Spanner	Relationalを含むMulti-modelの分散DB。	強い整合性と分散Transaction、大きな規模への拡張が重要な業務データ。	配置、遅延、Schema・Queryの設計、費用、移行互換性を評価する。単に「大規模だから」と決めない。公式概要
Firestore	Document型DB。	Document単位で状態を扱うWeb・Mobileなどのアプリ。	アクセスパターン、Index、Transaction、EditionやAPI、権限・Securityのモデルを確認する。公式概要
Bigtable	Row keyを持つKey-value／Wide-column型。	大量の時系列など、Keyに沿った低遅延・高Throughputの読書き。	Row keyとデータ分布、必要な検索、更新のまとまり、負荷を検証する。表形式という理由で一般的なRDBと同等に扱わない。公式概要

候補	データモデル	主に考える用途	選択を左右する条件
BigQuery	Tableとして扱うデータを、分析向けのColumnar Storageなどで処理する。	大量データの横断集計、BI、Ad hoc分析。	データ鮮度、取込み、Queryの範囲、同時実行、費用、Governanceを確認する。公式概要

例えば「受注の確定と在庫引当」は正しい更新のまとまりと応答時間、「全拠点の一年分の売上集計」は広い範囲の分析が中心です。同じデータを扱っていても、必要な処理を一つのDBへ集めるか、業務更新と分析を分けるかは設計判断になります。BigQueryにも更新やTransactionの機能はあるため、「更新できない」と覚えず、頻繁な小さな業務更新の主系に使うなら、その遅延・実行方式・費用が要件に合うかを検証します。

Index：検索を速めるための別の構造

Indexは、条件に合う行へ到達するための構造を追加します。適切なIndexで読込み量を減らせる一方、更新時にはIndexも維持するため、書込みや保存領域に負荷がかかります。複数Columnの順序、条件の選択性、Sort、取得件数などにより効果は変わります。「遅いQueryには全部Indexを付ける」という判断は避けます。

性能調査では、同じ時間帯の遅いQuery、実行回数、実行計画、読込み量、LockやI/Oの待ち、DB全体の負荷を組み合わせます。一回だけ遅いQueryと、一回は短いが大量に呼ばれるQueryのどちらが全体へ強く影響するかも確認します。変更は実際に近いデータと負荷で比較し、書込みへの影響も検証します。

Transaction：複数操作のまとまりを守る

Transactionは、複数の処理をまとまりとして確定・取消しする仕組みです。例えば受注と在庫引当が片方だけ成功すると、業務上の不整合が生じます。ACIDという用語では、Atomicity（全体をまとめて扱う）、Consistency（定義した制約を守る）、Isolation（同時実行時の見え方を制御する）、Durability（確定した結果を保持する）を区別します。

Isolationが強いほど常に高速という関係ではありません。同時更新の競合や待ちが増える場合があり、要件に応じて設計します。また、DB内のTransactionが正しくても、外部の決済APIまで同じTransactionに自動的に入るわけではありません。処理途中の失敗やRetryに備え、重複防止、処理状態、補償の方法を考えます。

3. 整合性・Replication・Cacheの関係

ここでの整合性は、更新が別の読取りにどのように見えるかという意味でも使います。TransactionのACIDのConsistencyとは、議論している対象が異なる場合があります。面接では「更新直後の利用者に最新の在庫が必要」のように、業務の条件へ置き換えて説明すると明確です。

Replication（複製）は、データを別のInstanceや場所へ反映する仕組みです。同期方式では、決められた複製側の応答などを確定条件に含め、非同期方式では後から反映します。遅延、可用性、未反映データの扱いに違いがあります。非同期Replicaを読む構成では、更新直後に古い値が見える可能性を業務が許容できるか確認します。具体的な保証はサービスと構成に依存します。

Cacheは繰り返しの読取りや計算を減らせますが、期限、無効化、更新反映、Hit率、個人情報や権限別データの分離を設計します。Cacheを導入するなら、遅延が改善する条件に加え、古い値を見せる条件を説明します。障害時に全要求がDBへ戻るCache missの集中にも備えます。

解説 | 理解を深める：Read replicaで解決する条件

Read replicaは、読取りを分担できる場合に性能改善の候補になります。一方、書込みの集中、長いLock待ち、非効率なQuery、接続管理の問題をすべて解消するわけではありません。読み書きの振分け、Replicaの遅延、障害時の動作を設計し、最新値が必要な操作の接続先を決めます。Replicaを増やした事実ではなく、利用者の遅延や処理成功率が必要条件を満たしたかで評価します。

4. HAとBackupは守る失敗が違う

図 06　**稼働系の複製と、過去へ戻す保護を分ける**



役割の違いを示す比較です。Replicaへの誤操作反映は方式に依存します。Backupも、誤削除前の世代、保管先と変更権限の分離、復元条件を満たして初めて復旧に使えます。別コピーがあるだけで安全とは限りません。

HA（High Availability）は構成要素が故障してもサービスを継続・回復しやすくする設計です。Cloud SQL for PostgreSQLのHA構成は同じRegion内のPrimary・Secondary zoneを使い、障害時にFailoverする仕組みを持ちます。Region全体の障害への対応や、誤った更新からの回復は別に検討します。Failover中の再接続など、アプリ側の動作も確認します。公式：About high availability

Backupは、誤削除、破損、望まない変更などに対して、過去の状態へ戻す材料です。同じ誤削除が複製先へ反映されるなら、Replicationだけでは元の状態を守れません。保存期間、保存先、復元権限、暗号鍵、復元先の容量とNetwork、復元手順を含めて計画します。

Cloud SQLではBackupやPoint-in-time recovery（PITR：指定時点への復旧）に関する機能・条件を確認します。取得済みであることと、必要な時点へ必要な時間内に復元できることを分けます。保持するBackupや関連Logが復旧に必要な条件を満たすかを確かめます。公式：Cloud SQL backups overview

5. 復旧目標を業務から決める

RTOは復旧までに許容する時間、RPOは失ってもよいデータの時間的な範囲を表す目標です。例えば「朝のBackupがある」は、直近の受注を失ってよいかへの回答になりません。「DBが起動した」は、利用者が業務を再開できるかへの回答にもなりません。復旧には検知、判断、環境の用意、復元、整合性確認、接続切替、業務確認

を含めます。

DR（Disaster Recovery）は、想定する大きな障害からの回復計画です。別Regionへデータを置くだけでなく、アプリ、Identity、Secret、DNS、経路、監視、操作権限、復旧担当と連絡方法を用意します。RTO/RPOはサービス名から保証できないため、復旧訓練で測定し、未達なら手順や構成を見直します。

6. OCI経験からの橋渡し

OCIでのBlock、Object、File、DB、Backupに関する経験を使い、アクセス方法と復旧条件からGoogle Cloudの候補を整理できます。ただし、DBエンジンが違えばSQL互換性、機能、運用、Replication、ライセンスなどの確認が必要です。既存製品の名前を置き換えるだけで移行可否を判断せず、Applicationの依存を洗い出します。

7. 小さなケース：注文を誤削除した

> 学習用の創作例です。誤った更新で一部注文が消えました。DBは稼働しており、Replicaも正常です。担当者はReplicaへ切り替えれば戻ると考えています。

まず、変更の時刻と範囲、継続中の更新、業務影響を確認し、被害の拡大を抑えます。Replicaに削除が既に反映されていれば、切替では戻りません。戻せる時点と復元方法を確認し、必要なら別環境へ復元して欠損内容を比較します。全体を過去へ戻すと、誤削除後の正しい注文を失う可能性があるため、復元範囲と整合性を業務担当と合意します。復旧後は注文数だけでなく、関連する決済・在庫なども照合します。

8. よくある落とし穴

- Storageの容量、IOPS、Throughput、Latencyを同じものとして扱う。
- ReplicaがあるのでBackupは不要と考える。
- Backup成功Logだけで、復元可能性と復旧時間を保証する。
- HA構成を、どのRegion障害や誤操作にも対応する構成として説明する。
- CacheやReplicaを増やす案で、古い値を読む業務影響を扱わない。

9. 面接での短い説明例

> 「データ基盤は、アクセス方法、Transaction、最新値の必要性、性能、復旧目標から選びます。HAやReplicationで故障への耐性を高めても、誤操作から戻すためのBackupは別に必要です。構成の採用後は、通常の性能だけでなく、Failoverと復元を実際の業務経路で確認し、RTO/RPOを満たすか測定します。」

10. 自己確認

1. 画像の保存と、在庫を引き当てるTransactionで、保存先に求める条件はどう違いますか。
2. Read replicaを増やしても改善しない性能問題を二つ挙げてください。
3. 毎日Backupが成功している顧客に、復旧準備として次に何を確認しますか。

解説 | 確認の観点を開く

1：データの単位、検索・更新方法、同時更新と業務制約、耐久性、復元条件を比較します。2：書込みの集中、非効率なQuery、Lock、接続管理などが候補です。3：許容損失と停止時間、復元先、権限・鍵・依存サービス、手順、実測した復元時間、復元後の整合性確認を確かめます。

IAM・Security：主体、権限、境界を設計する

認証と認可、IAMの継承、ワークロードのIdentity、Secret、監査、責任分界を一つの運用として捉える。

この章の到達点と早見表

Securityを「Firewallを閉じること」だけに限定せず、Identity、権限、通信、データ、Secret、変更、監査の組合せとして説明します。アクセス障害では、誰がどの資源へ何をしようとしたのかを具体化してから、必要最小限の修正を検証します。

- 認証は主体を確かめ、認可はその主体に操作を許すか判断します。
- Principalはアクセスする主体、RoleはPermissionのまとまり、Resourceは操作対象です。
- 親で付与した権限の継承を含め、実際に使える権限を確認します。
- サービスアカウントの権限と、そのアカウントとして動ける権限を分けます。
- 鍵やSecretは保存場所だけでなく、取得、更新、失効、監査まで管理します。

1. 認証と認可を分ける

Authentication（認証）は「誰として要求しているか」を確かめる処理です。Password、証明書、Token、連携するIdentity Providerなどが使われます。Authorization（認可）は、その主体がその対象にその操作を行ってよいかを判断する処理です。Loginできることと、全データを読めることは同じではありません。

人がConsoleを使うときと、アプリがAPIを呼ぶときでは、実際の認証主体が異なる場合があります。開発者本人の権限で成功しても、本番アプリのIdentityで成功する証明にはなりません。障害の再現では、実行環境、Credentialの取得元、Tokenの期限や対象、実際のPrincipalを確認します。Tokenや秘密鍵の原文をチャットへ貼り付けて確認する方法は取りません。

2. IAMの基本モデル

図 07　**誰かを確認し、何を許すかを判断する**



論理上の確認順序です。Policyの実装処理を厳密に描いたものではありません。IAM上の許可があっても、NetworkやService固有の条件などが残るため、実際の呼出しまで検証します。

Principal（誰が）＋ Role内のPermission（何を）＋ Resource（どれに）＋ 必要ならCondition（どの条件で）

Permissionは個別の操作に関する権限、Roleは関連するPermissionをまとめたものです。Roleの名前だけで判断せず、必要な操作を満たすか、余分な操作を許していないかを確認します。Google CloudのIAMは、Resourceに対する細かな認可を管理します。Allow policyでRoleとPrincipalを関連付けますが、Denyなど別の制御も実効的なアクセスに影響します。公式：IAM overview

最小権限は、仕事に必要な権限を必要な対象と条件に絞る考え方です。「一番弱そうなRoleを選ぶ」という意味ではありません。実施する業務を特定し、必要な操作を満たすRoleを、可能な限り狭い範囲へ付与します。運用できることを検証したうえで、定期的に使われていない権限や変更後の過剰権限を見直します。

3. Resource hierarchyと継承を理解する

Google CloudではOrganization、必要に応じたFolder、Project、その中のService resourceという階層を使って管理します。上位に設定したPolicyが下位へ影響するため、Projectだけ見て権限の全体を判断しません。例えば組織全体の共通担当と、個別アプリの運用担当をどの階層で分けるかが設計対象になります。公式：About resource hierarchy

本番と検証、部門、責任者、Securityの要件などから管理単位を考えます。分ける目的を明確にしないと、Projectを増やしても権限や変更管理が整理されないことがあります。逆に、共通の広い権限を親へ付与すると、意図しない対象まで操作可能になる場合があります。誰が作成、閲覧、変更、削除できるかを業務ごとに確認します。

解説 | 理解を深める：Allowがあれば必ず通るとは限らない

実際のアクセスは、認証の成功、IAMの許可、DenyやCondition、組織の制約、サービス独自の制御、Network到達性などの条件に左右されます。Allow policyで必要なRoleを見つけても、それだけで成功を保証しません。逆に、子ResourceのAllowを消しただけでは、親から継承したAllowが残る場合があります。変更前後の実効的な権限

と、対象操作の実行結果で確認します。

4. サービスアカウントはアプリのIdentity

Service accountは、主にアプリや計算ワークロードが使うアカウントです。実行環境に関連付けたService accountなどを使うことで、アプリの操作主体を人から分離できます。そのアカウントにRoleを与えることと、別の主体がそのアカウントを利用・偽装して動けるようにすることは、別の権限です。公式：Service accounts overview

サービスアカウントを利用できる主体が、そのアカウントの強い権限を使えるなら、間接的に大きな影響を与えられます。アプリごと・環境ごとの責任を区別し、広い権限のアカウントを多くのワークロードで共有する必要があるか見直します。Defaultという名前が、用途に合う最小権限を保証するわけではありません。

長期のサービスアカウント鍵は、漏えいすると外部から権限を使われる経路になり得ます。利用する実行基盤に適した短期CredentialやIdentity連携を優先して検討し、鍵が必要な事情がある場合は、保管、配布、利用者、更新、失効、検知を管理します。どの方式でも、アカウント自体に必要以上のRoleを与えないことは共通です。

5. Secret、暗号鍵、Tokenを混同しない

SecretはPasswordやAPI keyなどの機密値を指す言葉です。暗号鍵はデータの暗号化・復号や署名などに使います。Tokenは一定の主体や権限を示して要求を行うために使われます。いずれも「値を隠せば終わり」ではなく、発行者、保管先、取得主体、使用先、有効期間、失効と監査を整理します。

Secret ManagerではSecretへのアクセスをIAMで管理します。Secretを一元管理しても、取得するアプリに過剰な権限があれば影響は広がります。必要なSecretと環境に絞り、実行環境に合うCredentialの取得方式を検討します。公式：Secret Manager best practices

Rotation（更新）では、新しい値の作成だけでなく、利用側への反映、旧値との切替、確認、旧値の失効を計画します。旧値を即時に無効化すると停止する場合があり、長く残すと漏えい時の影響を残します。通常の更新と、漏えいが疑われる緊急の失効を分けて手順を決めます。SecretをLogやErrorに書かない設計も必要です。

6. Securityは層と責任分界で考える

Networkの制御は到達できる範囲を絞り、IAMは操作主体と対象を絞り、TLSは通信を保護します。データの暗号化、脆弱性対応、監査、Backup、Incident対応も組み合わせます。ある層を強くしたから別の層が不要になるとは限りません。例えばPrivateな接続でも、不正な権限を持つ内部主体の操作は別に制御します。

責任分界では、Googleが管理する基盤と、利用者が設定・運用するIdentity、データ、アプリ、Guest OSなどを、サービス単位で確認します。マネージド化は担当範囲を変えますが、顧客側の責任を一律に消しません。要件に対して何を誰が設定し、誰が監視し、異常時に誰が判断するかを明示します。

7. 監査Logは「誰が何をしたか」を調べる材料

Cloud Audit Logsは、管理操作やデータへのアクセスなどに関する監査を支えます。Logの種類、対象サービス、収集設定、保存期間、閲覧権限を確認してから調査に使います。Data Access audit logsは多くのサービスで明示的な有効化が必要で、例外もあります。「Logにないのでアクセスしていない」と結論付ける前に、記録される条件を確かめます。公式：Cloud Audit Logs overview

監査では、発生時刻、Principal、対象Resource、操作、結果、送信元などを対応させます。Logの出力時刻と要求時刻、タイムゾーン、検索期間も確認します。監査Log自体に機密情報が含まれる場合があるため、収集・共有は調査に必要な範囲に絞り、閲覧権限と保存先を管理します。

8. 権限エラーを調べる順序

1. 業務影響と、いつから何ができなくなったかを確認します。
2. 実行した操作、対象Resource、実際のPrincipal、失敗時刻、応答を特定します。
3. 認証が成立しているか、期待したCredentialを使っているかを確認します。
4. 必要なPermissionと実効的なPolicyを照合し、継承、Deny、Conditionなどを確認します。
5. IAM以外のService設定やNetwork制御なども、観測に応じて候補へ加えます。
6. 必要最小限の変更を行い、対象操作の成功と、許可すべきでない操作が引き続き制限されることを確認します。

9. OCI経験からの橋渡し

OCIでのユーザー、グループ、Policy、ワークロードのIdentityの経験から、主体・操作・対象・条件を整理する習慣を使えます。Google Cloudでは、Role、Policyの継承、サービスアカウントの利用権限などのモデルを改めて確認します。Policy構文や管理階層を機械的に対応させず、実際に許したい業務と実効権限から説明します。

10. 小さなケース：開発者は成功し、本番だけ403

> 学習用の創作例です。開発者が手元で実行するとObjectを読めますが、同じコードを本番へ出すと403になります。

最初に「同じコードだから同じ権限」と考えず、手元と本番のPrincipal、Credential、BucketやObjectの指定、要求時刻を比較します。本番が想定したサービスアカウントを使っていないなら、認証元の設定が候補です。Principalが正しければ、必要な読取り権限と実効Policyを確認します。403の発生層も確認し、IAMだけに候補を固定しません。

暫定的にOwnerなどの広いRoleを付けると、対象外の変更や削除まで許す可能性があります。必要なPermissionと付与範囲を特定し、テストできる環境で最小の修正を確認します。復旧後は権限の追加理由と有効な範囲を記録し、不要な一時権限が残らないようにします。

11. よくある落とし穴

- 認証成功を、必要な操作への認可成功と同じものとして扱う。
- 利用者本人の権限で再現し、本番アプリのIdentityを調べない。
- 権限が足りない理由を確認せず、最上位で広いRoleを付ける。
- Secretの保存先を変えただけで、取得権限やRotationを設計しない。
- 監査Logの未収集や検索失敗を、操作がなかった証拠にする。

12. 面接での短い説明例

> 「まず実際のPrincipal、対象Resource、操作、失敗時刻を特定し、認証と認可を分けて確認します。必要なPermissionを、親からの継承や制約を含む実効Policyと照合します。修正は必要な範囲へ絞り、業務の復旧と過剰なアクセスが増えていないことを検証します。SecretやIdentityの管理、監査、責任分界も運用として整えます。」

13. 自己確認

1. Loginできるのにデータを読めない状況を、認証と認可を使って説明してください。
2. サービスアカウントに与えた権限と、それを利用できる主体の権限を分ける理由は何ですか。
3. 監査Logに該当操作が見つからないとき、結論を出す前に何を確認しますか。

解説 | 確認の観点を開く

1：本人確認は成功していても、対象操作に必要なPermissionがない場合があります。2：利用可能な主体がサービスアカウントの権限を間接的に使えるため、両方の付与範囲を管理します。3：対象LogとService、収集設定、時刻・検索範囲、閲覧権限、保存・転送設定、検索自体の成功を確認します。

観測から事実を作る：Metrics・Logs・Traces

顧客の申告を測定可能な事実へ変え、数値の範囲・分母・確かさを説明できるようにする。

> **この章の到達点：**「ログを確認します」で止めず、どの問いにどの証拠を使い、何が分かり、何がまだ分からないかを説明する。製品の画面名より、調査したい問いを先に置く。

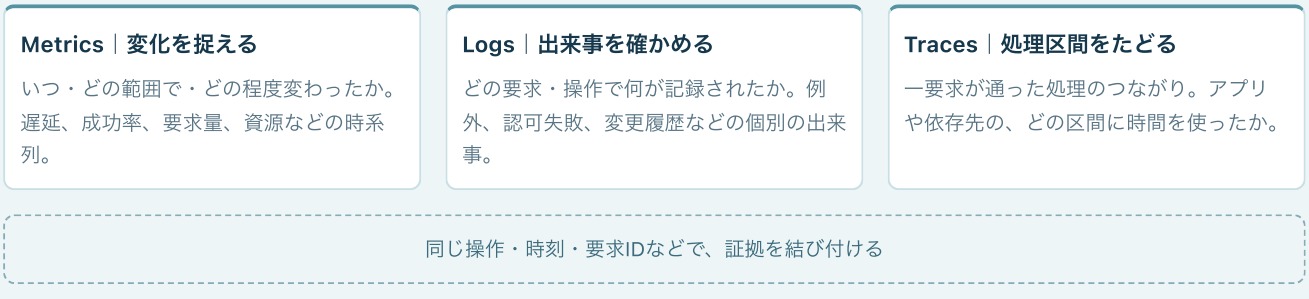
1. 観測は「利用者に起きたこと」と「内部で起きたこと」をつなぐ

Monitoring（監視）は、あらかじめ決めた指標や条件で状態を把握する活動である。Observability（可観測性）は、システムの出力から内部の状態を推定し、想定外の問いにも調査できるようにする性質として考えるとよい。両者を厳密な別製品と覚える必要はない。TAMに必要なのは、顧客の「受注が進まない」を、対象操作、時間帯、成功率、待ち時間、依存先の状態へ結び付けることである。

申告は調査の出発点だが、実測と同じ扱いにはしない。「お客様によると約10秒」「アプリログで確認した処理時間は8秒」「ブラウザーからの全体時間は未測定」のように記録する。同じ10秒でも、ログイン開始からなのか、検索ボタン押下からなのかで調査対象が変わる。測定区間を合わせないまま数値を比較すると、正常な差を障害と誤認する。

2. 三種類の証拠を、答えたい問いで使い分ける

図 08　問いに合わせて、三種類の証拠を重ねる



三つは補完する証拠であり、Metrics→Logs→Tracesという固定の調査順序ではありません。同じ対象操作・時刻・request ID／trace IDなどで対応付け、記録・計装・サンプリングの範囲も確認します。

証拠	主に答える問い	例と限界
Metrics：時系列の数値	いつ、どの範囲で、どの程度変化したか	要求数、エラー率、CPU、キュー長。集約により個別事象は見えにくい。
Logs：個々の出来事の記録	どの操作で何が起きたか	例外、認可失敗、変更履歴。記録しなかった事象や除外されたログは見えない。
Traces：一要求が通った処理のつながり	どの依存先・処理区間に時間を使ったか	アプリからDB・外部APIまでのSpan。計装・伝播・サンプリングに依存する。

Google CloudではCloud Monitoringで数値の傾向を、Cloud Loggingで記録されたイベントを、Cloud Traceで分散した要求の処理を調べる。Traceは収集された遅延データを使ってサービス間の待ち時間を調べるもので、導入しただけで全処理の原因を自動確定するわけではない。どこを計測しているか、依存先までTraceの文脈が渡っているかを確認する。公式：Cloud Traceの概要

構造化ログでは、時刻、severity、操作種別、リソース、結果、request IDなどを機械的に検索できる形で持つ。Cloud LoggingのjsonPayloadに入る構造化データは、対象フィールドを条件に調べられる。自由文に顧客名や秘密情報を大量に書く設計ではなく、必要な識別子と適切な権限・保持方針を組み合わせる。公式：構造化ログ

3. 最初のダッシュボードは利用者の結果から考える

Four golden signals（四つの主要な兆候）は、Latency（処理時間）、Traffic（要求量）、Errors（失敗）、Saturation（処理能力の逼迫）である。CPUだけを見て「正常」と判断するより、利用者の遅延と失敗、その時の要求量と資源の余力を組み合わせる方が、仮説を絞りやすい。成功応答と失敗応答の遅延も分ける。即座にエラーを返した要求が増えると、全要求の平均時間だけは短くなることがある。公式：Monitoring Distributed Systems

業務システムなら、HTTPの成功だけでなく「注文確定まで完了した割合」も必要になる。HTTP 200でも空の結果や業務エラーなら、利用者の目的は達成していない。一方、すべての4xxをサービス障害と数えると、意図した入力拒否まで失敗に含める可能性がある。何を良い結果と定義するかを、利用者とサービス責任者に確認する。

4. 平均とパーセンタイルの違い

平均は全測定値の合計を件数で割った値である。p50は並べた測定値の中央付近、p95は概ね95%の観測がその値以下となる境界、p99は概ね99%の境界である。p99は「99%の利用者が遅い」という意味ではない。要求単位の統計なら、同じ人の複数要求も別に数えるため、利用者割合とも一致しない。

> **学習用の創作例：**1,000要求のうち950件が100ms、50件が2,000msだったとする。平均は(950×100＋50×2,000)÷1,000＝195msである。生データを昇順に並べ、順位を切り上げて選ぶ方法ならp95は100ms、p99は2,000msになる。「平均約0.2秒」は、遅い50件の体験を十分に伝えない。

実際の監視サービスでは、全生データではなくHistogram（値の範囲ごとの件数）からパーセンタイルを推定する場合がある。Cloud MonitoringのDistribution型もbucketにまとめるため、bucketの幅や分布により精度が変わる。画面のp99を、全要求を完全に保存した厳密値と決めつけない。公式：パーセンタイルと分布値指標

複数サーバーのp95を単純平均しても、サービス全体のp95にはならない。全体の分布を作れるデータ、各bucketの件数、適切な集約方法を確認する。比較時には、測定区間、成功・失敗、操作種別、時間窓、地域、版、サンプル数もそろえる。大量の軽い検索が増えれば、重い検索が変わらなくても全体の平均値は改善する。

5. エラー率は分母まで言う

エラー率を説明するときは「エラー件数÷対象要求件数」を一組にする。「エラーが100件」は、全体100件なのか100万件なのかで影響が違う。対象期間も必要で、1分間の急増と1日の合計を混ぜない。分母にLBへ届いた要求だけを使う場合、DNS失敗などLBまで到達しなかった利用者は含まれない。観測地点の外側に残る空白を意識する。

retryがあると、一つの業務操作から複数の技術要求が発生する。サーバーへの試行120件のうち20件が失敗し、100個の業務操作はすべて完了したという状況もあり得る。試行単位の失敗率と業務操作単位の成功率は、どちらも意味があるが別の指標である。対応の効果を評価するときは、分母を途中で変更して改善に見せない。

6. 調査を一つの筋道にする

1. **範囲を固定：**操作、開始時刻とタイムゾーン、利用者・地域、通常時、業務影響を押さえる。
2. **入口を観測：**利用者から見た成功率と遅延が変わったかを確認し、全体か一部かを分ける。
3. **時系列を重ねる：**要求量、エラー、資源、変更イベントを同じ時間軸で比較する。
4. **代表要求を追う：**request IDやtrace IDでログとTraceを関連付け、遅い区間を絞る。
5. **反証する：**その仮説が正しければ何が見えるかを先に述べ、正常な要求とも比較する。
6. **次の証拠を選ぶ：**判断を変える情報を優先し、必要なチームへ範囲を限定して依頼する。

時刻が似ているだけでは因果関係は確定しない。変更直後に遅くなったなら有力な手掛かりだが、同時の負荷増加や外部依存も残る。ログが存在しないことも、処理が存在しなかった証明とは限らない。収集停止、保持期限、検索条件、権限不足を確認する。TAMは証拠の不足を明示しつつ、重大影響の緩和と専門チームの調査を並行させる。

解説 | OCI経験から移す考え方

OCIでのメトリック・ログ・監査・サポート証跡の扱いは、観測目的を整理する土台になる。ただし、Google Cloudでの自動収集範囲、ゲストOSの計装、ログ種別、権限、保持や課金は別に確認する。「OCIで自動的に見えたから同じ項目が見える」とは仮定しない。面接では共通原理を述べ、Google Cloudで確認する差分を具体化すればよい。

7. 短い口頭説明

> 「最初に、どの操作でどの程度の業務影響があり、いつから変化したかを確認します。利用者側の成功率と応答時間を基準に、Monitoringで範囲と時系列、Loggingで具体的なエラーや変更、Traceで処理区間を調べます。平均だけでなく遅い要求の分布を確認し、顧客申告と実測、相関と原因を分けます。影響が大きい場合は、安全な緩和と調査を並行します。」

8. 自己確認

解説 | ① CPUが低いのに遅い。次に何を見るか？

CPU平均だけでは余裕は分からない。入口の遅延・失敗、要求量、特定ノードや単一コアの偏り、DBのロック・接続待ち、I/O、外部API、キューを候補にする。Traceや待ち時間の記録から時間を使った区間を選び、やみくもに全ログを集めない。

解説 | ② p95が改善したら、全顧客の問題は解消したか？

断定できない。操作や地域ごとの偏り、残る遅い要求、エラー率、測定対象の変化を確認する。最終的には対象業務の回復条件を満たしたかで判断する。

解説 | ③ Traceに遅い要求が見つからない。どう説明するか？

未発生と断言せず、サンプリング、計装範囲、対象時間、伝播、検索条件を確認する。顧客側の要求時刻とIDを手掛かりに別の証拠と突き合わせる。

性能を考える：遅延・負荷・待ち行列と安全な緩和

遅い理由を資源不足だけに決めつけず、処理区間と待ちの仕組みから改善案を比較する。

> **この章の到達点：**「スケールすればよい」と即答せず、何が詰まっているか、どの対策が効く条件か、副作用をどう確認するかを説明する。

1. 「速さ」には違う種類の指標がある

Latency（遅延）は一つの要求が終わるまでの時間、Throughput（処理量）は単位時間に完了する仕事の量である。Concurrency（同時実行数）は同時に処理中・待機中の仕事の数として、どの範囲を数えるか定義する。たくさん処理できることと、一件が速いことは同じではない。まとめて処理するBatchは総処理量を上げても、個々の待ち時間を長くする場合がある。

一要求の遅延を、通信・処理・待ちに分ける。通信には往復回数や転送量、処理には計算や検索、待ちには接続プール、ロック、実行枠、キューがある。資源を使っている時間より、順番を待っている時間が長いこともある。そのためCPUが低いという事実だけでは、利用者が速く操作できるとは言えない。

利用者の待ち時間 ＝ 入口までの通信 ＋ 入口の待ち ＋ アプリ処理 ＋ 依存先の待ち・処理 ＋ 結果の通信

ただし並列に走る処理の所要時間をすべて足すと、全体時間を過大評価する。要求の完了を遅らせるCritical path（完了までの主要経路）を確認する。TraceのSpanが重なっているなら、その関係を読む必要がある。

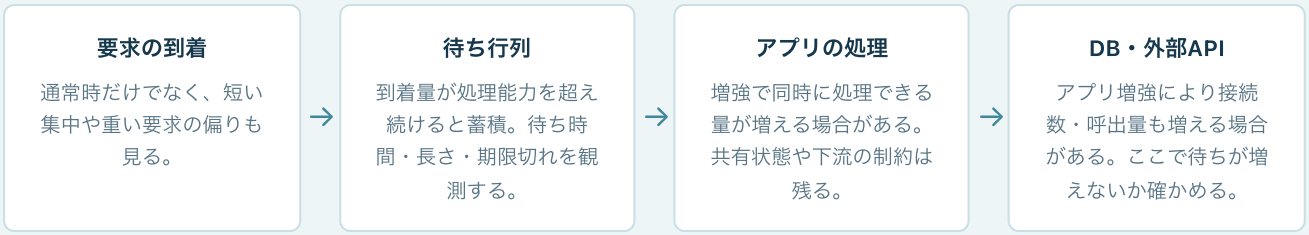
2. 目標は、業務と測定条件をそろえて置く

「普段2秒」は比較の基準であり、「必ず2秒に戻さないと業務継続できない」という条件ではない。どの操作を、どの利用者・負荷・データ量で、どの程度の割合で、何秒以内に終える必要があるかを合意する。重要業務を継続できる暫定目標と、平常状態への改善を分ければ、早期の影響緩和を選びやすい。

Google Cloudの性能設計指針も、要件、設計、観測、改善を継続する活動として整理している。製品のサイズを決めて終わりではなく、実際の利用状況が変わったときに目標と構成を見直す。公式：性能最適化の柱

3. 待ち行列は、上限に近づくほど問題になる

図 09　**到着量・待ち行列・下流の容量を、一つの流れで見る**



待ち行列を置くシステムの単純化した例です。到着超過が続くと未処理の仕事が蓄積します。キューを広げても処理能力は増えません。アプリを増強する場合も、DB・外部APIの接続数や呼出量、実際の待ちを合わせて確認します。

到着する仕事を処理しきれない時間が続くと、未処理の仕事が蓄積する。平均的に到着量が処理能力以下でも、短いBurst（集中到着）や重い要求の偏りで待ちが増える。CPU使用率が100%になる前に応答時間が悪化することがあるため、待ち時間・キュー長・期限切れも確認する。

> **学習用の創作例：**一秒に100件を安定して完了できる処理に、一秒120件が到着し続けると、単純化したモデルでは毎秒20件ずつ未処理が増える。キューを大きくするだけでは、到着超過を解消できない。待つ場所を増やした結果、期限に間に合わない仕事を長く保持する可能性がある。

Queue（待ち行列）は、短い負荷の波を吸収し、受け付けと処理を分離するために役立つ。注文受付を耐久性のあるキューへ保存し、Workerが後から処理する構成なら、瞬間的な需要と処理能力を切り離せる。ただし、受付成功と業務完了は別である。最大待ち時間、再処理、重複、順序、失敗を保管する場所、利用者への進捗表示を決める必要がある。

解説 | 平均同時実行数の小さな計算

安定した系を十分長く観測し、同じ境界で測るとき、平均的な系内件数は到着率×平均滞在時間で関係付けられる。例えば完了量が毎秒120件、平均滞在時間0.5秒なら、平均系内件数は約60件である。これは待ちが増え続ける状態や瞬間最大値をそのまま予測する式ではない。サーバーのCPUコア数や製品の同時実行設定と同じ数とも限らない。

4. 対策はボトルネックに対応させる

選択肢	効きやすい条件	確認したい副作用
Scale up：一台を強くする	単一ノードのCPU・メモリー等が制約	再起動、上限、費用、単一障害点が残ること

選択肢	効きやすい条件	確認したい副作用
Scale out：台数を増やす	仕事を分割でき、状態と依存先を扱える	DB接続・外部API負荷、状態同期、立ち上がり時間
Query・コード改善	不要な処理、検索計画、呼出回数が原因	結果の正しさ、別操作への影響、変更リスク
Cache：結果の再利用	同じ読取が多く、一定の古さを許容	無効化、機密データ、失効時の集中アクセス
非同期化・流量制御	完了を少し待てる業務、集中負荷	遅延、重複処理、順序、再試行、可視性

例えば、アプリを増やしたらDB接続上限に達することがある。アプリ一台あたりの接続プール上限x台数を見て、DB側で許容できる範囲と比べる。逆に、DB接続数が多いだけで不足と断定せず、実際に接続獲得待ちや拒否が発生しているか確認する。TAMはアプリ・DB・基盤チームの証拠を合わせ、局所改善が全体悪化にならないようにする。

Autoscaling（自動的な規模調整）には、負荷検知、判断、起動、アプリ準備の時間がある。VM、GKE、Cloud Runで調整対象と責任範囲が異なるため、「自動なので急増を必ず吸収できる」とは説明しない。アプリの状態、最小・最大規模、依存先の容量、必要な余力を確認する。公式の改善指針を起点に、実際の対象サービスで設定と挙動を確認する。公式：性能の継続的な観測と改善

5. TimeoutとRetryは、障害を広げることもある

Timeout（待つ上限時間）は、利用者の期限と資源占有を制御する。長くすれば遅い成功を待てる場合がある一方、待機要求や接続が積み上がり、全体の処理能力を下げることがある。短すぎると正常な処理を途中で諦め、再試行が増える。外側の業務期限と、内部の各呼出しの待ち時間を対応させ、期限切れ後に処理が残るかも確認する。

Retry（再試行）は一時的な失敗から回復する方法だが、負荷で失敗している系へ追加要求を送ると、悪循環になる。複数の層がそれぞれ再試行すれば、最下流の試行数は増幅する。再試行可能な失敗を選び、回数と総時間を制限し、待ち間隔を伸ばすBackoffと、同時再試行を分散するJitterを考える。障害時の扱いは、依存先が返す情報とサービス仕様に従う。公式：Handling Overload

「Timeoutした」は「処理が一度も実行されなかった」ではない。決済・注文・作成要求を再送するなら、Idempotency（同じ要求を繰り返しても意図しない重複結果を起こさない性質）を確認する。識別キーや重複排除、処理状態の照会などが必要になる。TAMとしては、データの整合性を犠牲にした速さの回復を提案しない。

6. 緩和と恒久対策を分け、安全に試す

安全な緩和の候補には、直近変更の切り戻し、重い非必須機能の一時停止、入口での流量制限、正常な系統への誘導、容量追加などがある。いずれも自動的に正解ではない。切り戻し可能性、データ互換性、別拠点の余力、承認者、実行者、停止条件を確認し、実施後は顧客の業務結果まで見る。

あるノードの障害で残りのノードへ負荷が寄り、それらも落ちるのがCascading failure（連鎖障害）の一例である。健康判定で外す行為自体は正しくても、残りの処理能力が足りなければ悪化する。負荷を受ける総量、再試行、復帰直後の集中まで考える。公式：Addressing Cascading Failures

恒久対策は、どの仮説を検証した結果かを明示する。「CPU増強をしたら改善した」だけでは、CPUが唯一の根本原因と証明できない場合がある。再現可能な条件で一要因ずつ比較し、資源、応答時間、成功率、費用の変化を記録する。原因が未確定なら、確認済みの効果と残る不確実性を分けて報告する。

7. 負荷試験は平均的なアクセス数だけでは足りない

本番に近い操作比率、データ量、利用者の地域、読み書き比率、接続数、Warm／Cold状態、急増、長時間運転を考える。軽い読取だけで通った試験を、重い検索や大量更新の保証には使わない。通常負荷で目標を満たすかに加え、限界を超えたときの拒否・縮退・回復も見る。試験自体が本番や外部サービスへ影響しない範囲を決める。

性能改善の成功条件は、対象操作の遅延と失敗が目標に入り、他業務と費用に許容できない悪化がなく、負荷が変化しても観測できること、といった形で置く。特定の一回が速くなっただけで、繁忙期の安定性まで確認済みとは扱わない。

8. 短い口頭説明

> 「業務上必要な応答時間と影響範囲を確認し、利用者側から遅延の区間を分けます。要求量、待ち行列、資源、依存先を見て、容量不足なのか、ロックや通信などの待ちなのかを絞ります。緊急時は安全な切り戻しや負荷制御を検討し、スケールする場合もDB接続など下流の制約を確認します。対策後は、同じ操作と負荷条件で遅延・成功率・業務回復を検証します。」

9. 自己確認

解説 | ① キューを大きくすれば、処理能力不足は解決するか？

継続的な到着超過は解消しない。短いBurstを吸収する用途と、処理能力の不足を分ける。待ちの上限、期限切れ、Workerの拡張、入口の制御を考える。

解説 | ② Timeoutを長くしてエラーが減った。成功と判断してよいか？

業務期限、待ち時間、同時要求、接続占有、後続への影響を確認する。遅い成功が増えただけなら、業務上の改善になっていない可能性がある。

解説 | ③ Cacheを入れる提案で、先に聞くべきことは？

同じ結果を再利用できるか、どの古さまで許されるか、利用者ごとに結果が違うか、更新・失効時にどう扱うかを確認する。鮮度や情報保護の要件が選択を変える。

信頼性とSRE：SLO・Error budget・HA・DR・改善

信頼性を「落ちないこと」から、利用者の目標・回復条件・継続的な改善へ広げる。

> **この章の到達点：**SLI／SLO／SLAを区別し、Error budgetを計算し、HA・DR・Backupを障害シナリオと業務要件から説明する。高い数値を提案する前に、測定と運用が成立するかを考える。

1. 信頼性は、利用者が必要な結果を得られること

サーバーが起動していても、注文確定に失敗したり、期限内に処理が終わらなかったりすれば、利用者にとって良いサービスではない。信頼性を議論するときは、対象のUser journey（利用者が目的を達成する一連の操作）を特定する。全機能を同じ重要度にせず、停止で業務が止まる操作、少し遅れてもよい処理、後で取り戻せる処理を分ける。

SRE（Site Reliability Engineering）は、信頼性を人の頑張りだけに依存させず、測定、ソフトウェアによる改善、変更管理、学習の仕組みで扱う考え方である。TAMはSRE担当者を代行するというより、顧客の業務目標と技術・運用上の課題をつなぎ、適切なチームと改善を進める立場で説明する。

2. SLI・SLO・SLAを一つの例で分ける

用語	何を表すか	学習用の創作例
SLI：サービス水準指標	利用者に提供したサービスの測定値	対象検索のうち、正しい結果を2秒以内に返した割合
SLO：サービス水準目標	SLIについて合意する目標と評価期間	直近30日で、その割合を99.9%以上にする
SLA：サービス水準合意	対象、測定方法、責任、未達時の扱い等を定める合意	契約でサービス範囲と補償条件を定める

SLOを「必ずSLAより高くする」という暗記で済ませない。契約上のSLAと、顧客アプリのSLOで測定範囲が異なることがある。クラウド基盤のSLA達成は、アプリの業務目標達成をそのまま保証しない。依存先、アプリ、認証、ネットワーク、運用を含めた利用者の体験を、別に設計・測定する。

SLIには、何を良い要求とするか、分母に何を含むか、どこで測るか、どの期間で評価するかが必要である。意図した入力エラー、利用者のキャンセル、メンテナンス、無通信の時間帯をどう扱うかも定義する。都合の悪い失敗を後から除外してはいけない。Google SREのSLO導入ガイドは、利用者に意味がある指標を選び、関係者と目標を共有して改善する進め方を示している。公式：Implementing SLOs

3. Error budgetは、目標までに許容する不良の量

Error budget（エラーバジェット）は、完全に良いサービス100%とSLOの差を、対象期間・対象イベントの単位で表した余地である。故障させてよいという許可ではなく、信頼性と変更・機能開発の判断を、同じ測定値で話し合うために使う。

> **学習用の創作例：**評価期間の対象要求が1,000,000件で、成功率SLOが99.9%なら、不良要求の予算は1,000,000×(1−0.999)＝1,000件である。既に不良が400件なら、使用率は400÷1,000＝40%、残りは600件、残率は60%となる。ここでは評価期間の最終的な総要求数が既知という単純化をしている。実運用の途中では、期間や分母の変化を考慮する。

時間ベースの別例では、30日を連続した43,200分として測り、その99.9%を良い時間と定義するなら、不良時間の予算は43.2分である。これは要求ベースの1,000件を43.2分へ変換したものではない。繁忙時間に大量の要求が失敗した1分と、利用が少ない1分では、要求ベースの影響は異なる。Request-basedとWindow-basedのSLIも区別し、何を一単位として数えるかを必ず確認する。

Burn rate（消費速度）は、不良率が許容不良率の何倍かを見る考え方である。SLO99.9%に対し直近の不良率が1%なら、1%÷0.1%＝10倍である。一瞬の揺れと継続的な消費を区別するため、短い窓と長い窓を組み合わせて判断する。低トラフィック時の数件や観測欠損にも注意する。

> **予算を使い切ったら、機械的にすべての変更を止めるとは限らない。**事前に合意した方針に従い、新機能より信頼性改善を優先するか、変更範囲を絞るかを判断する。重大な脆弱性修正や復旧のための変更まで止めると、かえって危険になる。SLOの対象、測定の妥当性、顧客影響、責任者をそろえて決める。

4. HA・DR・Backupは、守る失敗が違う

図 10　RPOはデータの時間幅、RTOは業務復旧までの時間



復旧データ時点→障害発生は、失い得るデータの時間幅です。障害発生→業務復旧は、対象業務の回復までの時間です。それぞれRPO・RTOの目標以内かを確認します。本図では障害発生をRTO計測の起点とし、発見時刻と混同しません。図上の実績値と、合意した目標値は区別します。図の間隔は時間の長さを表しません。

HA（High Availability：高可用性）は、一定の障害が起きてもサービスを継続しやすくする設計である。DR（Disaster Recovery：災害・重大障害からの復旧）は、通常の冗長化では吸収しきれないシナリオから業務を回復する計画と能力である。Backupは過去時点のデータを復元するための保護で、レプリカが存在することと同じではない。

Zoneの障害を想定して別Zoneへ冗長化しても、同じRegion全体や、誤った更新・削除が同時に反映される失敗には別の対策が必要になる。レプリケーションは可用性や復旧を支えるが、誤操作やデータ破損も複製し得る。復元可能な世代、分離された保管、変更権限などを考える。クラウドの管理サービスを使っても、アプリの接続先切替や業務確認が顧客側に残る場合がある。

RTO（Recovery Time Objective）は、どれだけの時間で回復する必要があるかという目標である。RPO（Recovery Point Objective）は、どこまで過去へ戻ったデータを許容できるかという時間軸の目標である。「RPO15分」は、必ず15分で復旧するという意味ではない。何をもって業務回復とするか、どのデータに適用するかを明確にする。公式：Disaster recovery planning guide

> **学習用の創作例：**受注業務のRTOが2時間、RPOが15分とする。データを10分前の整合した時点へ戻せても、認証・接続先・アプリ・帳票連携の確認に3時間かかればRTOを満たさない。毎15分のBackup予定があっても、失敗・転送遅延・整合性の問題があればRPO達成を保証できない。

5. 復旧構成は、費用と実行できる運用を合わせて選ぶ

Cold、Warm、Hotといった復旧構成は、待機側の準備度を表す整理として使う。平常時に用意する資源が少なければ、復旧時の構築やデータ復元に時間を要する傾向がある。常時稼働する待機系は回復を早めやすいが、費用、同期、構成差分、切替後の整合性を扱う運用が必要である。名前だけでRTO／RPOが決まるわけではない。

復旧テストでは、Backupジョブの成功だけでなく、取り出せること、復元できること、アプリが使えること、データが整合していることを順に確認する。暗号鍵、権限、証明書、DNS、ネットワーク、外部依存、必要な容量、担当者不在時の権限も確認する。切替後に元へ戻すFailbackの手順と、二つの系へ同時に書いてしまう危険も扱う。

nozのOCI経験からは、責任分界、メンテナンスの影響確認、Support・Engineeringとの連携を思考の土台にできる。一方、Google Cloudのサービスごとに、冗長化の単位、切替方式、データ保護、顧客操作の範囲を確認する。HAオプションの名前が似ていても、停止時間やデータ損失の条件まで同等とは扱わない。

6. 変更リスクを、観測できる小さい単位へ分ける

変更前に、対象、期待する効果、失敗の兆候、確認時間、切り戻す条件と手順を決める。Canary（カナリア）は、一部の対象へ先に変更を適用して、正常な群と比較しながら影響を検知する方法である。少量にしただけで安全になるわけではなく、代表性のあるトラフィック、成功率と遅延、DBなど共有部分への影響を見る。公式：Canarying Releases

Rollback（切り戻し）は、アプリの版を戻せば必ず完了するとは限らない。Schema変更、データ書込み、外部APIの変更は戻しにくい場合がある。前後の版が共存できるか、データ互換性があるか、戻すより前進修正した方が安全かを担当者と判断する。TAMは実行を約束する前に、担当と判断権限、顧客への説明内容をそろえる。

7. 復旧後は、責めるより再発と影響を減らす

Postmortem（障害の振り返り）では、影響、検知、緩和、回復までの時系列と、何が機能し何が不足したかを記録する。個人の注意力だけを原因にすると、別の人が同じ状況に置かれたときの再発を防ぎにくい。誤操作を許した権限、検証不足、監視、手順、複雑さ、連携の遅れを含めて考える。公式：Postmortem Culture

改善は「手順を周知する」で終えず、何を変え、誰が担当し、何をもって確認するかを決める。例えば「削除前に復旧可能性を自動確認する」「異常な成功率で変更を止める」「顧客向け状況更新の担当と間隔を決める」のように、再発確率・検知時間・回復時間・影響範囲のどれを改善するか対応させる。推測の原因と確認済みの事実は分ける。

8. Toil削減は、自動化の数ではなく継続的な負担で測る

Toilは、本番運用で繰り返され、手作業的で、規模に応じて増え、長期的な価値を残しにくい作業として捉える。「嫌な仕事」や「すべての会議」の同義語ではない。毎回同じ状態確認や定型復旧を人が実施するなら、設計で不要にできるか、自動化できるか、例外だけ人が判断する形にできるかを考える。公式：Eliminating Toil

自動化も誤りを高速で広げる可能性がある。読取・判定・変更を分け、対象範囲、冪等性、試行環境、承認が必要な操作、実行記録、停止と復旧の方法を設計する。効果は、削減できた手作業時間だけでなく、誤操作、例外対応、保守時間、顧客待ち時間まで見る。noz自身の経験を話す際は、調整・提案した範囲と、他チームが設計・実装した範囲を分ける。

9. 短い口頭説明

> 「利用者の重要な操作を決め、その成功率や応答時間をSLIとして測り、期間を付けたSLOを顧客と合意します。Error budgetを、変更と信頼性改善の優先順位を話し合う材料にします。構成は想定する障害とRTO／RPOから選び、HA、レプリケーション、Backupを組み合わせます。復旧テストで業務まで戻せることを確認し、障害後は仕組みと運用を改善して、再発と顧客影響を減らします。」

10. 自己確認

解説 | ① SLO99.95%、対象200万要求なら、不良要求の予算は？

2,000,000×(1−0.9995)＝1,000件。評価期間、良い要求の定義、分母が確定していることが条件である。サービス契約の補償条件とは別に扱う。

解説 | ② 別Zoneにレプリカがあれば、Backupは不要か？

不要とは言えない。Zone障害と、誤更新・削除・破損は異なる。保護したい失敗、過去時点への復元、保管の分離と権限を確認する。

解説 | ③ RTOを短くしたい顧客に、最初に何を確認するか？

どの業務をどの状態へ戻す必要があるか、現在の回復にどこで時間を使うか、停止損失、予算と運用体制を確認する。起動時間だけでなくデータ・認証・接続・業務検証を含める。

構成と移行の判断：顧客価値・制約・トレードオフ

製品名から提案を始めず、顧客の成功条件と制約から選択肢を比較し、段階的な実行と検証まで説明する。

> **この章の到達点：**「どの製品が最適か」にすぐ飛びつかず、判断基準、代替案、選んだ理由、変える条件を説明する。Google Cloudの全サービスを覚えることより、必要な原理を適切な製品と責任分界へ結び付ける。

1. 要件は、目的・成功条件・制約へ分ける

「クラウド化したい」「マイクロサービスにしたい」「高可用性にしたい」は、まだ技術方針や希望である。その先の目的を聞く。繁忙期に受注を落とさない、変更を早く届ける、夜間作業を減らす、老朽化を解消するなど、何を改善したいかで構成は変わる。

成功条件は、対象業務と測定方法を含める。「性能を上げる」なら、対象操作、負荷条件、応答時間の分布、失敗率を定める。「運用を楽にする」なら、月あたりの手作業、夜間呼出し、変更時間などで確認する。数値がまだなければ、無理に補わず、顧客と基準を決める行動を述べる。

制約には、予算、納期、変更可能範囲、技術スキル、既存契約、データの所在、接続先、停止許容、組織の判断手順がある。「政府系だからすべて同じ」とは扱わず、対象顧客・データ・サービスに適用される具体的な条件を確認する。nozの業界理解は、法令名を多く挙げることより、制約が設計と運用をどう変えるかへ結び付けると伝わりやすい。

2. 一本の要求の流れと、依存するものを描く



図では、要求とデータがどこへ流れるか、誰の管理か、どこに状態があるかを示す。線を引いたら、接続性、認証、失敗時の挙動を確認する。管理画面への接続、CI/CD、監査ログ、復旧操作など、利用者の通常要求には見えない経路も重要である。

依存先は台数より重要なことがある。アプリを三つのZoneへ置いても、共有DB、認証基盤、外部API、共通設定の配布で停止する可能性が残る。単一障害点を探すときは「一台だけ」を探すのではなく、「これが失敗すると重要業務が止まるもの」を探す。

3. 比較の軸は、同じ条件でそろえる

Google Cloud Well-Architected Frameworkは、運用、セキュリティ、信頼性、費用、性能、持続可能性という観点で設計を検討する。今回の面接準備では、顧客の目的に合わせ、特に性能・信頼性・セキュリティ・費用・運用負荷の関係を説明できるようにする。これは学習の整理であり、Googleの面接採点表を意味しない。公式：Well-Architected Framework

観点	確認すること	ほかの観点への影響例
性能	対象操作、負荷、遅延、処理量	容量追加は費用を増やし、Cacheは鮮度を変える。
信頼性	障害シナリオ、SLO、RTO／RPO	冗長化は費用と同期・切替の運用を増やす。
セキュリティ	データ、Identity、境界、鍵、監査	統制を運用できる権限設計と手順が必要になる。
費用	通常・ピーク・転送・保管・運用を含む総額	安価な資源でも、停止損失や保守負担が大きい場合がある。
運用負荷	監視、変更、復旧、スキル、担当体制	管理サービスで減る仕事と、新たに必要な設計がある。

すべてを最高水準にする回答は、顧客が実行できないことがある。必須条件を満たせない案を先に除き、残った案を優先する価値で比較する。得点表を使うなら、重みを顧客と合意し、根拠のない数字で精密に見せない。「この条件ではAを選ぶが、負荷や運用体制が変わればBを再検討する」と条件付きで説明する。

4. 管理サービスを選ぶときも、残る責任を示す

VM、Container、Serverlessの選択では、既存アプリの互換性、OSや実行環境の制御、常時稼働か要求駆動か、状態、拡張、運用スキルを比較する。OS制御が必要な既存アプリならVMが現実的なことがある。Containerを採用しても、必ず大きなKubernetes環境を運用する必要があるとは限らない。実行要件と管理したい範囲から選ぶ。

DBでも、運用負荷だけでなく、データモデル、Transaction、整合性、検索、移行互換性、拡張、復旧を確認する。Managedにすれば、Schema、Query、権限、データ分類、業務テストまで不要になるわけではない。ベンダーが担う部分と、顧客が設定・検証する部分を、サービス単位で説明する。

Regional／Multi-regionという名前も、それだけで業務継続を保証しない。データの配置、書込みの扱い、通信遅延、切替、バックアップ、依存先の範囲を確認する。冗長化と回復の設計は、利用者の目標、想定する失敗、実際に試験できる能力へ結び付ける。公式：信頼性の柱

5. 費用は、利用量と単価と設計で分解する

クラウド費用の増加を調べるときは、まず期間、対象Billing account・Project・サービス、通貨、割引や調整、比較条件をそろえる。値上がりを直ちに疑う前に、要求量、稼働時間、資源サイズ、保管量、保持期間、データ転送、再試行、ログ量などに分ける。見積りは前提を付け、価格・契約・地域の現行条件を確認する。

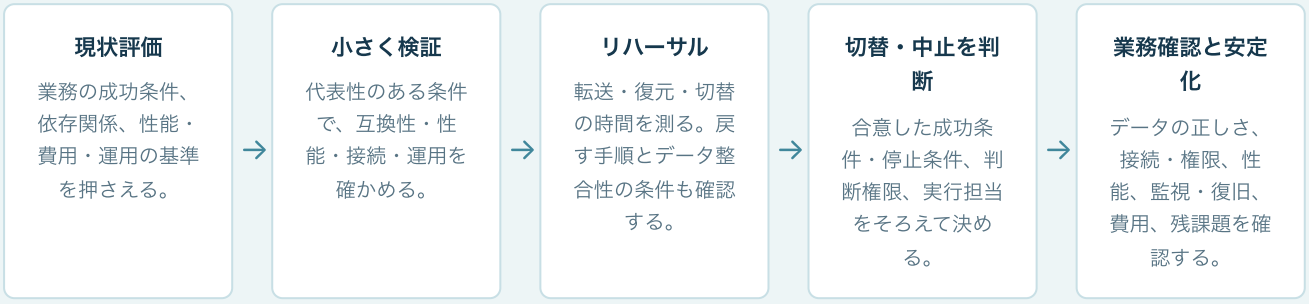
費用削減では、使わない資源の停止、適切なサイズ、需要に応じた拡張、保管方式と保持期間、転送経路、無駄な処理を見直す。長期の利用約束による割引は、需要が安定しているか、契約条件を満たすかを確認して比較する。目先の単価低下で、使わない容量や変更しにくさを抱える可能性もある。Google Cloudの費用指針も、支出を事業価値に合わせ、継続的に見直すことを中心に置く。公式：費用最適化の柱

「予算を設定したから、それ以上は絶対に請求されない」と説明しない。2026年10月の公式文書では、通知中心のAlerts-only budgetsと、対応サービス向けのSpend cap budgetsを区別している。通知だけの予算設定は自動上限ではない。上限機能を使う場合も、対応サービス、適用範囲、計測・反映、停止が業務へ及ぼす影響を確認する。公式：予算と予算通知

> **学習用の創作例：**月額が増えていても、受注数が二倍、費用が1.3倍なら、受注一件あたりの費用は改善している可能性がある。一方、受注が変わらずログ転送だけが増えたなら別の改善余地がある。「総額を下げる」と「単位業務あたりの効率を上げる」を区別し、顧客が何を必要としているか確認する。

6. 移行では、現状を知る前に着地点を固定しない

図 11　**段階ごとに検証し、進める・修正する・止めるを判断する**



各段階で検証結果を受けて計画を見直します。切替・中止条件と判断者を事前に合意し、切替後の書込みを含めて戻せる条件を確認します。DNSを戻すだけでデータも安全に戻るとは限りません。

移行の評価では、アプリ・DB・データ・接続・運用を棚卸しし、依存関係と業務のまとまりを調べる。Rehost（大きく作り替えず移す）、Replatform（一部を管理サービス等へ置き換える）、Refactor／Re-architect（コードや構成を見直す）などの方針は、期限、互換性、価値、変更リスクで比較する。全面的な刷新が常に最短でも安全でもない。

Google Cloud Migration Centerの移行計画も、発見・評価と基盤の準備を踏まえ、依存関係を考えて移行のWaveや一緒に動かす群を決める。既存接続、Identity、監視、課金、セキュリティの土台を、最初の本番切替までにそろえる。公式：移行計画の概要

1. **評価：**現状の性能・費用・障害・運用を把握し、成功条件と依存関係を決める。
2. **小さく検証：**代表性があり、失敗時の影響を制御できる対象で互換性と運用を確かめる。
3. **移行計画：**順序、データの移し方、停止、差分同期、切替・中止条件と責任者を決める。
4. **リハーサル：**実測した転送・復元・切替時間で計画を修正し、戻せる条件を確認する。
5. **切替と検証：**技術指標だけでなく業務の正しさ、接続、権限、監視、費用を確認する。
6. **安定化：**旧環境の扱いを決め、最適化と運用改善を続ける。

データの切替では、最後の書込み、整合性、二重更新、切替後の変更をどう戻すかが難所になる。「DNSを戻せばRollbackできる」とは限らない。書込みが新環境へ進んだ後に旧環境へ戻すなら、どのデータが正か、差分をどう扱うかを計画する。移行成功と旧環境削除は別の判断にする。

7. 技術判断を、関係者の行動に変える

技術チームには、観測事実、仮説、再現条件、必要な支援を伝える。業務責任者には、影響、選択肢、必要な判断、見込みの確度を伝える。経営層には、停止・売上・生産性・リスク・投資の関係へ置き換える。相手に合わせて説明の深さを変えても、事実と未確認事項は変えない。

意見が割れるときは、誰が正しいかより、共有する成功条件と相違点を整理する。例えば運用側は夜間の負担、開発側は変更速度、業務側は繁忙期の停止、財務側は支出を重視しているかもしれない。選択肢、効果、リスク、追加費用、期限、実行者、承認者をそろえ、合意できないリスクは適切な責任者へ上げる。TAMだけで、開発納期や復旧時刻を確約しない。

8. 短い口頭説明

> 「まず顧客が改善したい業務と成功条件を確認し、予算、停止許容、データ、運用体制などの制約を整理します。要求とデータの流れ、状態、依存先、責任分界を押さえ、複数案を性能・信頼性・セキュリティ・費用・運用負荷で比較します。選んだ理由と残るリスクを示し、小さな検証から段階的に進めます。切替・中止条件と担当を合意し、実施後は業務結果と運用負荷まで確認します。」

9. 自己確認

解説 | ① 「高可用性にしたい」に対して、何を聞くか？

どの業務が、どの失敗で、どれほど止まると困るか、必要な回復時間とデータ損失許容、予算・運用体制を聞く。構成やSLOの数値を先に決めない。

解説 | ② 安いVMへ変更する提案で、何を検証するか？

対象負荷での性能、余力、障害時の振舞い、必要な運用、変更リスク、総費用を確認する。料金だけでなく、重要業務の成功条件を満たすかで判断する。

解説 | ③ 移行後にアプリが起動した。完了してよいか？

業務の正しさ、データ整合性、接続・権限、性能、監視、Backup・復旧、運用引継ぎ、費用、残課題を確認する。起動は移行成功の一部にすぎない。

nozの経験を、技術理解につなぎ直す

確認済みのBaseDB移行支援を、観測・仮説・副作用・責任分界の観点で読み直す。今後の確認項目は過去の実績へ混ぜない。

今回使う経験と、その根拠

ローカルのInterview Story・Answer Bankに、2026/10/26の本人説明・確認に基づくBaseDB本番移行支援が記録されています。以下は顧客や案件を特定しない要約です。今回、個別SR・メトリック原本は再照合していません。Google Cloud上で同じ対応を実施した経験としては扱いません。

> 本番移行でデータ取り込みが想定より進まないとの相談を受け、nozは顧客へSR起票とメトリック確認を依頼しました。CPU・メモリ使用率のスパイクを共有し、暫定的なリソース増強の妥当性をSR担当者へ確認しました。追加費用、処理の中断、再起動が必要なことを説明し、顧客が変更を実施しました。変更後にCPUの逼迫が改善し、取り込みと移行が完了したと本人が説明しています。

この経験が、どの技術論点につながるか

経験の要素	学ぶ技術・判断	説明時の境界
画面から確認できるメトリックを集めた	使用率、飽和、待ち、処理量、観測期間の違い	noz自身がOSコマンドを実行したとは言わない。
リソース増強を暫定案として相談した	ボトルネック、スケールアップの成立条件、効果の検証	CPU不足を根本原因と確定したとは言わない。
CPUとメモリが同時に増えた	複数変数の変更と、因果関係の限界	CPU増加だけに改善を帰属しない。
費用と処理中断・再起動を説明した	暫定策の副作用、時間制約、顧客の意思決定	停止時間、正式な承認経路、費用額を補わない。
SR担当者が確認し、顧客が操作した	専門家連携、責任分界、実施と調整の違い	DBチューニングや移行実装を自分の実績に加えない。

同じ状況をGoogle Cloudで考えるなら

まず、対象がCompute Engine上の自己管理DBか、Cloud SQLなどの管理型サービスかを確認します。リソース変更の方法、停止や再起動、変更できる範囲、接続や実行中処理への影響は、サービス・構成・操作に依存します。OCIの当時の挙動を、そのままGoogle Cloudへ当てはめません。Data章とCompute章で仕組みを理解し、該当製品の公式手順を確認して方針を具体化します。

これからの学習で加える観点

次は、処理の進捗量、CPU以外の待ち、I/O、ロック、並列度、実行計画、データ量、接続数、変更に必要な容量・割当、失敗時の戻し方、元のサイズへ戻す条件を考えます。これは今後の一般的な確認観点です。当該移行ですべて実施した事実ではありません。どれを先に確認するかは、残り時間と入手できる証拠によって選びます。

解説 | 面接で聞かれたら：「増強で改善したなら、CPU不足が原因では？」

「CPUの逼迫が改善し、取り込みが完了したことは確認しています。ただ、変更でCPUとメモリが同時に増えており、それぞれの寄与は分離できていません。私は当時の観測とSupportの確認を基に、時間制約下の暫定策として提案しました。根本原因の特定とは分けて説明しています。」

解説 | 自分の別の経験で練習するための六問

1. 顧客が達成したかった業務と制約は何か。
2. 何を観測し、何が申告・推測だったか。
3. 自分は何を比較・判断したか。
4. 他の人は何を判断・実行したか。
5. 何が改善し、どこまで証拠があるか。
6. 今なら何を追加で確認するか。それは当時の実施と区別できているか。

面接で伝える技術｜詰まった時の立て直しと回答の組み方

短い結論、根拠、条件、検証を使い、考えていることを相手が追える回答にする。

> この章は、今回の準備向けのコーチング提案です。Googleの公式採点基準や、合格率の改善を実証した手法ではありません。採用担当者から「回答を整理するため30〜60秒もらってよい」と案内された記録を踏まえています。

回答がすぐ出ない時も、判断の途中を見せられる

詰まった時に用意しておきたいのは、考える順序を相手へ伝える短い言葉です。仕様の細部が出てこなくても、顧客が達成したいこと、既知の事実、考えられる選択肢、その選び方を話せる場合があります。どこまで分かり、何を確認すると決められるかを示すことで、面接官が追加質問をしやすい回答になります。

ただし、原理を問われたら原理を説明します。「顧客影響を確認します」を毎回返しても、DNS、認証、レプリケーションなどの仕組みへの回答にはなりません。用語説明、仮想ケース、過去経験のどれを聞かれているかを先に判別します。質問の種類に合う短い型を選ぶことが、整って聞こえる第一歩です。

最初の一文で、回答の方向を伝える

仮想ケースでは「現時点ではAを優先します。理由はBです。ただしCなら判断を変えます」が使えます。結論、根拠、適用条件がそろうため、相手はどの判断を検討すればよいか分かります。すべてを言い終えるまで結論を隠す必要がありません。

> 「現時点では、Webサーバーの増強より先にDB待ちを確認します。Web側の処理時間は変わらず、DB区間だけ伸びたという前提だからです。ただし、その計測が別の時間帯なら、同じ要求のトレースで取り直します。」

「Aを優先します」は、他の可能性を永久に捨てるという意味ではありません。「まず何をするか」と「その後に何を残すか」を分けます。一方、知識の定義を聞かれた時は「認証は誰かを確認すること、認可はその主体に何を許すかを判断することです」のように直接答え、必要なら例を一つ添えます。

詰まった時の立て直しを、四つの動作に分ける

1. **問いを短く言い換える。**「このケースでは、復旧の優先順位をどう決めるか、という理解でお答えします。」意図が本当に不明な時だけ確認し、明確な質問を毎回聞き返しません。
2. **判断に必要な前提を一つ置く。**「データの正しさは保たれ、主な影響は遅延だと仮定します。」事実でない前提は仮定と明示し、後で新情報があれば修正します。
3. **分かる原理から候補を二つ程度に絞る。**「処理そのものが遅い場合と、処理を始めるまで待っている場合を分けます。」製品名が思い出せない時も、比較する対象を具体化できます。
4. **次の観測と判断の分岐を話す。**「DB待ちが伸びていればDB側を、待ちがなくアプリ内だけ伸びていればアプリ側を調べます。」結果が出た後に何を変えるかまで述べます。

この四つのうち必要な部分だけ使います。30秒の定義問題に長い枠組みを持ち込まず、障害ケースでは情報収集と安全な影響緩和を並行させます。言葉を整えている間も、顧客の問題を前へ進める判断を残します。

「賢そう」に聞こえる言葉が役立つ理由

次の表現は、難しい単語よりも判断の精度を伝えやすいものです。言葉を挿すだけで終えず、その直後に対象と証拠を続けます。

- **「事実として確認できているのは〜、仮説は〜です」**：情報の確かさが分かります。「CPUが上がった」は観測、「CPU不足が遅延の原因」は仮説、と分けます。
- **「同じ条件で比較します」**：測定の妥当性を示せます。操作、測定区間、負荷、時間帯など、何をそろえるかを具体的にします。
- **「判断を変える条件は〜です」**：柔軟性と一貫性を同時に示せます。反論を受ける前から、選択肢の成立条件を説明できます。
- **「暫定対処の完了条件と、恒久対策の完了条件を分けます」**：業務継続と再発防止の違いが伝わります。応答時間の回復と、根本原因の解消を混同しません。
- **「時間的な相関はありますが、原因は未確定です」**：変更直後の障害を手掛かりにしつつ、版ごとの比較や検証が必要だと示せます。
- **「その案の副作用として〜を確認します」**：単発の対処から依存関係へ視野を広げられます。増強なら下流接続、再試行なら重複と負荷、切替ならデータを挙げます。

整理時間は、短く目的を添えてもらう

使える言い方は「影響と選択肢を整理してからお答えしたいので、30秒ほどいただいてもよろしいでしょうか」です。今回の案内では30〜60秒の整理が可能とされていますが、毎問その時間を使い切る必要はありません。答えられる問いには直接答え、複数の制約を扱う時に使います。

考えている間は「目的・事実・仮説・判断・確認」の五語程度をメモする方法があります。文章を書き切るより、最初の結論と二つの根拠を決めます。整理後は「まず結論からお伝えします」と入り、考えた過程のすべてを再現しようとしません。

知らない仕様は、分かる原理と確認が必要な点へ分解する

「正確な既定値は今ここでは断定できません。設計上は、同時実行数と下流の接続余力を合わせて判断します。対象サービスの現行仕様を確認した上で上限を決めます」のように話します。知らない範囲を一か所に限定し、知っている仕組みと、その数値が判断へ与える影響を説明します。

原理そのものを知らない時は、その境界も短く伝えます。「その方式の動作原理までは説明できません。比較する上では整合性と遅延の条件を確認したいです」と言い、分かる範囲の判断へ接続します。曖昧な知識を長く話すより、次の質問が可能な回答にします。すべての仕様について同じ前置きを使うと内容が薄くなるため、確認が必要な点だけに付けます。

深掘りでは、最初の結論を守ることより前提を更新する

新情報で判断が変わったら、「その情報で優先順位を変えます。先ほどはAを前提にしましたが、今はBなのでCを優先します」と説明します。これは同じ情報で意見が揺れたのではなく、根拠が変わった結果だと伝える形です。

自分の誤りに気付いた場合も、訂正する箇所を特定します。「先ほどRPOを復旧時間と説明しましたが、正しくは許容するデータ損失の時間幅です。復旧時間の目標はRTOです。今回の案はその二つで評価し直します」と、訂正後にケースへ戻ります。長い弁解を足す必要はありません。

技術的な判断を、TAMの行動へつなぐ

「専門チームへ確認します」だけでは、何を任せ、何を自分で持つかが見えません。「私は影響、時系列、仮説、必要な証拠を整理し、DB担当へ待機と実行計画の確認を依頼します。顧客には次回連絡時刻と当面の緩和案を共有します」のように、依頼する技術課題と自分の責任を一緒に述べます。

過去経験では「私が実施したのはA、実装を担当したのはBです。私はCを判断してDを調整しました」と分けます。実装者でなくても、判断や調整の内容を具体的に説明できます。結果が未確認なら「顧客が復旧を確認したところまで」「再発率の継続測定はしていません」と、確認できた終点を示します。

短い回答の修正練習

次の回答を、自分ならどう直すか先に考えてください。長さを増やすより、判断の根拠と次の行動を足します。

解説 | 練習1：「状況によります。まずいろいろ確認します」

改善例：「まず支障の程度と時間制約を確認します。停止している場合に加え、継続中でも受注低下や締切への重大な影響があれば、安全な緩和と関係者の招集を優先します。影響が許容範囲なら、どの操作がいつから遅いかを確認し、普段と同じ区間で比較します。」条件と行動を対応させることで、一般論を具体的な判断へ変えています。

解説 | 練習2：「その製品は知らないので、専門家に聞きます」

改善例：「その製品固有の動作は確認が必要です。まず要求される整合性と停止許容を整理し、候補がそれを満たすか比較します。私は要件と未確認点をまとめ、専門担当へ具体的な仕様確認を依頼します。」専門知識の境界を示した上で、判断を前へ進めています。

解説 | 練習3：「変更直後なので、変更が原因です。戻します」

改善例：「変更起因を優先仮説にします。新旧版の失敗率とログで確認しつつ、旧版が健全でデータ互換性と戻し手順が確認できれば切り戻します。原因の検証と、安全な緩和の判断を並行します。」相関・原因・戻せる条件を区別しています。

面接で伝える技術｜20場面の言い回しと、その後に話すこと

答えが出ない、仕様を知らない、反論されたなどの場面で使える日本語フレーズ集。

> 以下は暗記用の完成答案ではなく、回答を立て直すための言い出しです。自分が説明できる内容と、そのケースの事実に合わせて使ってください。提案した言い回しがGoogleで評価されることや合格を保証するものではありません。

01｜答えがすぐ思い浮かばない

狙い：考える時間を取り、最初の一文を決める。

> 「影響と選択肢を整理してからお答えしたいので、30秒ほどいただいてもよろしいでしょうか。」

その後に話すこと：目的、既知の事実、最初の仮説をメモし、結論から再開します。全部を調べ尽くす段取りを作る必要はありません。

> **逆効果になりやすい例：**無言が長く続く／毎問60秒を使い切る／整理時間の後も質問を言い直すだけ。

02｜質問の意図が分からない

狙い：答える対象をそろえる。

> 「今回は、技術的な切り分けの進め方と、顧客への初動説明のどちらを中心にお答えするのがよいでしょうか。」

その後に話すこと：返答に合わせて焦点を決めます。二つとも必要なら「まず切り分け、その後に連絡の進め方をお話しします」と順番を示します。

> **逆効果になりやすい例：**明確な質問を毎回確認し直す／質問を別の得意分野へすり替える。

03｜ケースの前提が足りない

狙い：判断を変える不足情報だけを確認する。

> 「まず、どの業務にどの程度の支障があり、急ぐ必要のある締切や繁忙時間帯があるかを確認したいです。継続できていても重大な影響はあり得るため、影響の大きさと時間制約で緩和・連携の緊急度を判断します。」

その後に話すこと：得られた回答で優先順位を変えます。情報を得られないなら合理的な仮定を明示し、後で見直す条件を置きます。

> **逆効果になりやすい例：**構成、台数、型番などを目的なく大量に質問する／全情報がそろうまで行動を決めない。

04｜製品の正確な数値や既定値を知らない

狙い：仕様の未確認点を限定し、原理と判断を示す。

> 「正確な既定値は確認が必要です。設計上は、この制限がピーク時の同時処理を収められるかで判断します。」

その後に話すこと：必要な容量と余裕、下流への影響を話し、現行仕様と実測で確認する対象を具体化します。数値を答える問題なら未回答の範囲も明確にします。

> **逆効果になりやすい例：**それらしい数値を即答する／「ドキュメントを見れば分かります」で説明を終える。

05｜仕組み自体の理解が浅い

狙い：知っている範囲と不足を短く示す。

> 「その方式の内部動作までは説明できません。私が説明できるのは、利用側から見た整合性と遅延の判断までです。」

その後に話すこと：説明できる境界の中で原理と選択条件を話します。追加の推測を事実にせず、学習が必要な点を一つに絞ります。

> **逆効果になりやすい例：**用語を連ねて内部動作を知っているように見せる／何でも「基本的には」で断言する。

06｜技術用語や英単語が出てこない

狙い：意味の説明を先に進める。

> 「用語がすぐ出てこないのですが、同じ操作を繰り返しても、一度行ったときと同じ効果に保てる性質のことです。」

その後に話すこと：機能と具体例を説明します。用語を思い出したら「冪等性です」と補い、その言葉探しだけに時間を使いません。

> **逆効果になりやすい例：**似た別の用語で断定する／英単語を思い出すまで回答を止める。

07｜原因が一つに決められない

狙い：候補を絞り、見分ける観測を示す。

> 「現時点では、処理時間の増加と待ち行列の増加を分けます。同じ要求のトレースと待機時間で優先順位を付けます。」

その後に話すこと：二つ程度の候補について、何が見えたらどちらへ進むかを話します。仮説を多く挙げること自体を成果にしません。

> **逆効果になりやすい例：**「ネットワーク、CPU、DB、アプリ、全部あり得ます」で終える／根拠なしに一つへ固定する。

08｜二つの案から即答を求められる

狙い：暫定結論と採用条件を示す。

> 「今の条件ならAを選びます。停止時間を短くする優先度が高いからです。ただし費用の上限が厳しければBを再検討します。」

その後に話すこと：選んだ案の根拠と副作用を一つずつ述べます。比較を続ける必要があるなら、結論を変える追加情報を限定します。

> **逆効果になりやすい例：**「どちらにもメリットがあります」だけで選ばない／条件なしに「必ずA」と言う。

09｜面接官から反論された

狙い：反論が新情報か、説明不足かを確かめる。

> 「その制約を加えると、先ほどの案では難しくなります。Aを前提にしていましたが、Bを優先する案へ変えます。」

その後に話すこと：新情報なら前提を更新します。説明不足なら判断根拠を補います。相手の言い方に合わせるだけでなく、変わった理由を明確にします。

> **逆効果になりやすい例：**結論だけすぐ反転する／最初の回答を守るために提示された条件を無視する。

10｜途中で自分の誤りに気付いた

狙い：訂正する点を特定し、正しい説明へ戻る。

> 「一点訂正します。先ほどの説明はRTOとRPOを逆にしていました。復旧時間の目標がRTO、許容するデータ損失の時間幅がRPOです。」

その後に話すこと：訂正後の定義を使ってケースを評価し直します。結論が変わるなら、その箇所も明示します。

> **逆効果になりやすい例：**気付いた誤りをそのまま広げる／弁解を長く続けて本題へ戻れない。

11｜深掘りで分かる範囲を超えた

狙い：確かな部分を維持し、技術課題を明確に渡す。

> 「ここから先の内部実装は説明できません。私なら、想定する失敗時にどの保証が維持されるかを専門担当へ確認します。」

その後に話すこと：何を知れば設計判断できるか、質問と検証条件を話します。元の顧客要件と、回答後に自分がする判断を残します。

> **逆効果になりやすい例：**「専門家に任せます」で終える／分からない内部構造を推測で断定する。

12｜緊急障害で原因が分からない

狙い：原因調査と影響緩和を並行させる。

> 「原因はまだ未確定です。業務影響が大きいため、安全な迂回と関係者の招集を進めながら、変更と観測結果を確認します。」

その後に話すこと：緩和案の対象、副作用、戻せる条件、成功確認を述べます。原因確定を待つ必要がない行動と、証拠が必要な変更を区別します。

> **逆効果になりやすい例：**全ログがそろうまで待つ／原因不明なのに強い変更を無条件で行う。

13｜過去経験で自分の役割を聞かれた

狙い：自分の判断と他者の実行を具体化する。

> 「私は影響と時系列を整理し、対応の優先順位を合わせました。技術的な修正は担当Engineeringが実施しました。」

その後に話すこと：自分が決めたこと、依頼したこと、合意を得たことを具体例で示します。実装を担当していなければ、その分担のまま価値を説明します。

> **逆効果になりやすい例：**チームの成果をすべて自分の実装として話す／「調整しました」だけで具体的な判断を省く。

14｜成果の数値が手元にない

狙い：確認済みの結果と測っていない効果を分ける。

> 「定量的な改善率は測っていません。確認できた結果は、顧客が業務再開を確認し、合意した対応を完了したことです。」

その後に話すこと：実際に確認した終点、残る課題、後から測るなら何を使うかを話します。再発防止案と再発率の改善実績は区別します。

> **逆効果になりやすい例：**改善率や削減時間を推測で作る／結果が分からないまま「大成功でした」とまとめる。

15｜話が長くなった

狙い：結論と優先順位へ戻し、追加質問の余地を作る。

> 「論点が広がったので、結論を二点に絞ります。まずAで影響を抑え、次にBで原因を確認します。」

その後に話すこと：補足を一旦止め、理由を一つずつ述べます。「必要であれば検証の詳細を説明します」と区切り、相手の次の質問を待ちます。

> **逆効果になりやすい例：**「最後に」と何度も続ける／新しい枝の説明を追加して終点を遠ざける。

16｜抽象的すぎると感じた

狙い：一つの操作と観測へ落とす。

> 「例えば注文APIなら、成功率と応答時間を利用者側で測り、失敗した同じ要求をトレースでたどります。」

その後に話すこと：誰のどの操作を、何と比較するかを具体化します。一般原則は、その例を説明するための短い言葉にとどめます。

> **逆効果になりやすい例：**「包括的に監視し、最適化します」など、実際の観測や行動が分からない言葉を重ねる。

17｜OCIの経験をGoogle Cloudへつなげたい

狙い：共通原理と製品差を分ける。

> 「OCIでの経験から、障害単位と依存先を分けて考える点は使えます。Google Cloudでの実現方法は、サービスごとの範囲と保証を確認します。」

その後に話すこと：共通する考え方を先に説明し、該当するGoogle Cloudの候補と差を話します。具体仕様が未確認なら、その範囲を限定します。

> **逆効果になりやすい例：**似た名前のサービスを完全同等とする／Google Cloudの問いにOCIの製品名だけで答える。

18｜安全性と速度の両立を聞かれた

狙い：リスクを比較し、小さく確かめる。

> 「すぐに全体変更するより、影響範囲を限定して試し、成功条件を満たせば広げます。ただし継続被害が大きい場合は封じ込めを優先します。」

その後に話すこと：検証対象、停止条件、戻し方、待つことのリスクを示します。慎重さを無行動とせず、段階を分けて判断します。

> **逆効果になりやすい例：**「安全のため何もしません」／「緊急なので検証は不要です」と一括で扱う。

19｜費用やビジネスとのつながりを問われた

狙い：技術指標を顧客の成果へ翻訳する。

> 「応答時間の改善自体に加え、受注担当者が必要な件数を処理できるかで効果を確認します。増強費用もその効果と比較します。」

その後に話すこと：顧客が必要な処理量、停止許容、運用工数など、確認できる指標を選びます。売上改善など未計測の因果は作りません。

> **逆効果になりやすい例：**「ビジネス価値を最大化します」とだけ答える／技術改善から売上増加を根拠なく断定する。

20｜最後に一言でまとめるよう求められた

狙い：判断、理由、次の行動を残す。

> 「今はAを優先します。Bという影響を抑えるためです。Cを確認し、条件が違えば方針を見直します。」

その後に話すこと：新しい論点を増やさず、話した内容から最も大事な三点を選びます。過去経験なら自分の行動、確認できた結果、学びに置き換えます。

> **逆効果になりやすい例：**すべての技術項目をもう一度列挙する／まとめで新しい未説明の提案を出す。

使う表現を三つだけ選ぶ

まず「現時点では〜を優先します」「判断を変える条件は〜です」「確認できている事実と仮説を分けます」の三つから、自分に使いやすいものを選びます。ケース練習で一つ使い、その後に具体的な内容を続けられたか確認してください。多くのフレーズを一度に覚えるより、自分の判断につながる言葉を少数使える状態にします。

振り返りでは、表現の滑らかさだけでなく、質問へ答えたか、理由を示したか、条件が変わった時に修正できたかを確かめます。言い回しが整っていても、その後に対象・証拠・行動が続かなければ改善対象です。

ケース01｜「システムが遅い」を、業務と測定へつなぐ

顧客申告・平常時・業務上の必要条件を区別し、調査と影響緩和を組み立てる。

> 学習用の創作ケースです。実際のGoogleの出題・採点基準を再現したものではありません。数値や構成は、このケースの前提です。

出題：分かっていることと、まだ分からないこと

受注担当者から「クラウド上の検索が遅い」と連絡がありました。Google Cloud上にWebアプリとDBがあります。顧客は「普段は2秒程度、今は20秒くらい」と申告しています。発生範囲、エラーの有無、測定区間、構成の詳細は未確認です。

先に自分で考える

最初に何を質問しますか。その情報で何を判断し、どの調査へ進むかまで説明してください。

30〜60秒で考えを整理し、結論と理由を声に出してください。解説を読む前に、最初の質問と次の行動を決めます。

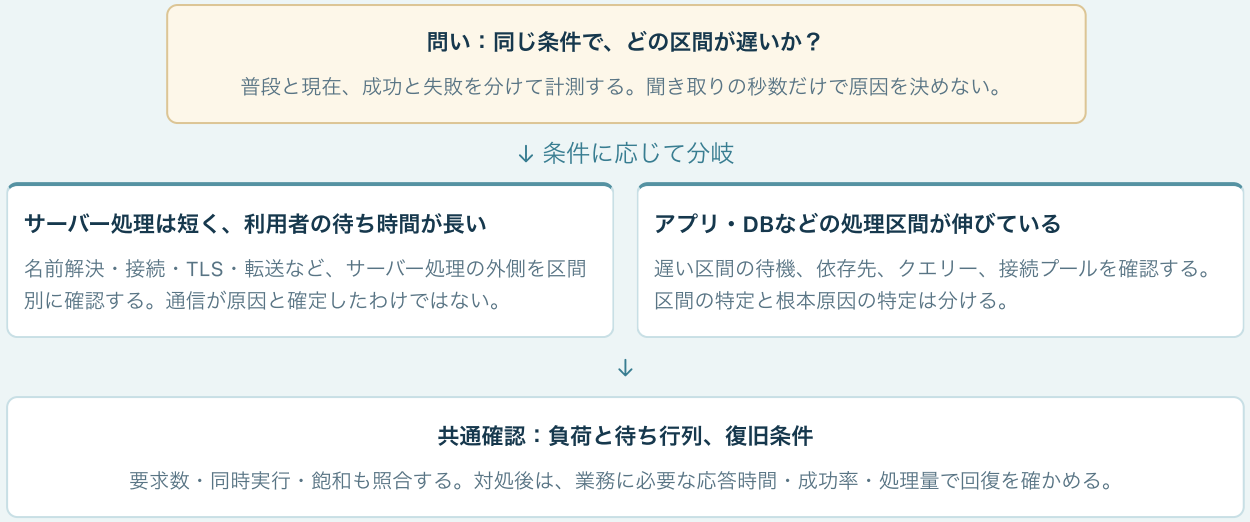
解説 | 解説を開く：判断の分岐と回答例

優先して確認する質問と、その理由

1. 「どの操作で、受注や電話対応にどの程度の支障が出ていますか。締切はありますか」：重大度と緩和を急ぐ必要性を判断します。停止だけを影響と考えません。
2. 「普段と現在の時間は同じ操作・区間ですか。待てば完了しますか」：比較の条件をそろえ、遅延と失敗を分けます。聞き取りの値は実測と区別します。
3. 「いつから、全員か一部か、直前に変更や負荷増加はありましたか」：時間・利用者・操作で切り分け、比較対象を選びます。

仮説と検証：結果によって次の行動を変える

図 12　遅延は、同じ要求のどの区間で増えているか



顧客申告は出発点です。同じ操作・時間帯・測定区間で確かめ、観測に応じて調査先を変えます。重大な業務影響があれば、安全な緩和と関係者連携を並行します。業務停止だけが重大影響ではありません。

1. 入口またはネットワークの遅延：利用者側の計測とサーバー側の処理時間を比べます。サーバー処理が短ければ、名前解決・接続・転送などの区間を分けて確認します。
2. アプリまたはDBの処理遅延：同じ時間帯のトレース、アプリ処理時間、DB待機や実行計画を確認します。DB区間が伸びていればクエリーと競合へ、アプリ内で待っていれば依存先や接続プールへ進みます。
3. 負荷や待ち行列：要求数、同時実行数、キュー、リソースの飽和を確認します。負荷と遅延が同時に増えても、相関だけで原因確定とはせず、健全な時間帯や小さな検証と比べます。

影響を緩和する：副作用も一緒に考える

重大な影響なら、調査と並行して業務の迂回、不要な重処理の一時停止、既知の安全な変更の巻き戻しを検討します。増強はボトルネックが改善する場合に絞ります。アプリを増やしてDB接続を急増させるなど、下流を悪化させる案は避けます。

復旧の確認と予防

同じ操作・測定区間で遅延とエラーを測り、顧客が必要とする業務処理量を回復したか確認します。平均だけでなく、遅い要求の割合も見ます。

予防では、主要操作の計測、変更履歴、繁忙期の容量確認、業務上の回復目標を整えます。通常2秒だったことと、業務継続に必要な時間は別々に記録します。

TAMとして持つ役割

TAMは影響、優先順位、仮説、担当、次回連絡時点を整理します。運用・開発・DB担当には必要な証拠を対象と時間帯付きで依頼し、原因未確定の段階で復旧時刻を約束しません。

口頭回答の例

> まず遅い操作と業務影響、急ぐ締切を確認します。普段2秒・現在20秒という値は顧客申告として扱い、測定区間と処理の成否をそろえます。その上で、入口の通信、アプリ・DBの処理、負荷による待ちを候補にし、同じ時間帯の計測で遅い区間を絞ります。重大な影響があれば、安全な迂回や既知の変更の巻き戻しを担当者と並行して検討します。回復は顧客の業務に必要な応答時間と処理量で確認し、残った平常時との差は継続調査します。

90〜150秒程度で説明するための練習用の例です。Google公式の回答時間制限ではなく、発話時間も実測していません。全文を暗記せず、判断を変える条件を自分の言葉で説明してください。

> 注意：「CPUを見ます」だけでは、なぜその情報が必要かが伝わりません。質問と観測が、次の判断につながる形で説明します。

公式: Google SRE — Effective Troubleshooting

条件を変えた追加問題

新しい情報：顧客は「検索が5秒以内なら通常人数で処理できます。普段は2秒でした」と説明しました。当面の回復目標をどう置きますか。

解説 | 追加問題の考え方を開く

当面は5秒以内で必要な処理量を維持することを、顧客と確認した上で緩和の目標にします。どの要求の割合で5秒を満たすか、エラーを許容するかも確認します。2秒は平常時との差を調べる比較基準として残し、5秒到達だけで原因解決や恒久復旧を宣言しません。

ケース02｜一部の利用者だけ接続できない

成功経路を比較対象にし、DNS・接続・TLS・HTTPを混同しない。

> 学習用の創作ケースです。実際のGoogleの出題・採点基準を再現したものではありません。数値や構成は、このケースの前提です。

出題：分かっていることと、まだ分からないこと

社内ネットワークの一部から業務サイトに接続できません。別拠点とモバイル回線では成功しています。画面には「接続できません」とだけ表示されます。Google Cloud側全体の障害か、顧客側の問題かは分かりません。

先に自分で考える

どの順番で情報を集め、成功する経路との違いをどう使いますか。

30〜60秒で考えを整理し、結論と理由を声に出してください。解説を読む前に、最初の質問と次の行動を決めます。

解説 | 解説を開く：判断の分岐と回答例

優先して確認する質問と、その理由

1. 「どの拠点・利用者・端末で失敗し、何時からですか」：影響範囲と共通点を特定します。成功者の存在だけで重要度を低くしません。
2. 「表示される正確なエラーと、名前解決・接続・TLS・HTTPのどこまで進むかは分かりますか」：失敗した層に合わせて調査を絞ります。
3. 「成功と失敗でDNS、プロキシ、VPN、送信元IP、直近の変更は異なりますか」：経路の差を検証対象にします。

仮説と検証：結果によって次の行動を変える

1. DNSの差：成功拠点と失敗拠点で名前解決結果・参照先を比べます。古い宛先や解決失敗が片側だけなら、DNS設定とキャッシュの経緯を調べます。

2. 通信経路やフィルタリングの差：正しい宛先にTCP接続できないなら、必要な方向・ポートの経路、FW、プロキシを調べます。片方向の許可だけで往復が成立するとは限りません。
3. TLSまたはアプリ側の差：接続は成立しても証明書検証やHTTPで失敗するなら、証明書、接続名、プロキシ検査、返されたステータスを確認します。HTTP応答があることは経路の一部が通った証拠ですが、業務の成功とは別です。

影響を緩和する：副作用も一緒に考える

承認された別経路や代替端末で業務継続できるか検討します。セキュリティ設定の全開放や証明書検証の恒久無効化は緩和策にしません。設定を戻す場合は差分、対象、戻せる範囲を確認します。

復旧の確認と予防

影響を受けた拠点と代表的な操作で再接続し、ログイン・検索・登録まで必要な処理を確認します。単にトップページが表示されるだけで復旧としません。

DNS・プロキシ・FWの変更管理、代表拠点からの外形監視、成功時の経路情報を整えます。複数拠点の比較ができれば、次回の切り分けを短くできます。

TAMとして持つ役割

TAMは顧客ネットワーク担当とクラウド側担当の間で、時刻・送信元・宛先・エラーをそろえます。担当組織を推測で決めず、どの境界まで成功しているかを共有します。

口頭回答の例

> 一部拠点だけ失敗するため、まず影響範囲と正確なエラーを確認し、成功する拠点を比較対象にします。名前解決、接続、TLS、HTTPのどこで止まるかを区別します。解決結果が異なればDNSを、接続できなければ経路やフィルタリングを、HTTP応答があればその内容とアプリ側を調べます。同時に安全な代替経路で業務を継続できるか確認します。復旧は失敗していた拠点の実際の操作で確かめます。

90〜150秒程度で説明するための練習用の例です。Google公式の回答時間制限ではなく、発話時間も実測していません。全文を暗記せず、判断を変える条件を自分の言葉で説明してください。

> 注意：成功拠点があることだけでGoogle側の原因を除外したり、pingの成功だけでアプリまで正常と断定したりしません。

条件を変えた追加問題

新しい情報：失敗拠点でもサイトのIPへのTCP接続は成功しますが、ホスト名でのTLS検証に失敗します。次は何を確かめますか。

解説 | 追加問題の考え方を開く

ホスト名と証明書の対象名、有効期限、信頼チェーン、時刻、途中のTLS検査を確認します。IPへの接続成功だけでは、SNIを含むホスト名でのTLSやHTTPが正しいとは言えません。証明書検証を無効化して業務を再開する判断はせず、適切な証明書や承認された経路へ修正します。

ケース03｜リリース直後に5xxが増えた

変更との時間的関係を手掛かりにしつつ、安全な巻き戻し条件を考える。

> 学習用の創作ケースです。実際のGoogleの出題・採点基準を再現したものではありません。数値や構成は、このケースの前提です。

出題：分かっていることと、まだ分からないこと

新しいアプリを公開した10分後から、外部Application Load Balancer経由の注文APIで5xxが増えました。全件ではなく、一部が失敗しています。新旧のアプリが混在中です。DBスキーマ変更の有無は未確認です。

先に自分で考える

最初の判断と、ロールバックの前に確認すべき条件を説明してください。

30〜60秒で考えを整理し、結論と理由を声に出してください。解説を読む前に、最初の質問と次の行動を決めます。

解説 | 解説を開く：判断の分岐と回答例

優先して確認する質問と、その理由

1. 「注文失敗の割合と影響金額、重複注文や処理結果不明はありますか」：業務とデータへの影響を把握します。再試行すればよいとは決めません。
2. 「どのバージョン・経路・時間帯で失敗し、何を変更しましたか」：新旧比較と変更差分を使って対象を絞ります。
3. 「旧版へ戻す手順は検証済みで、DBやメッセージとの互換性はありますか」：緩和の安全性を判断します。

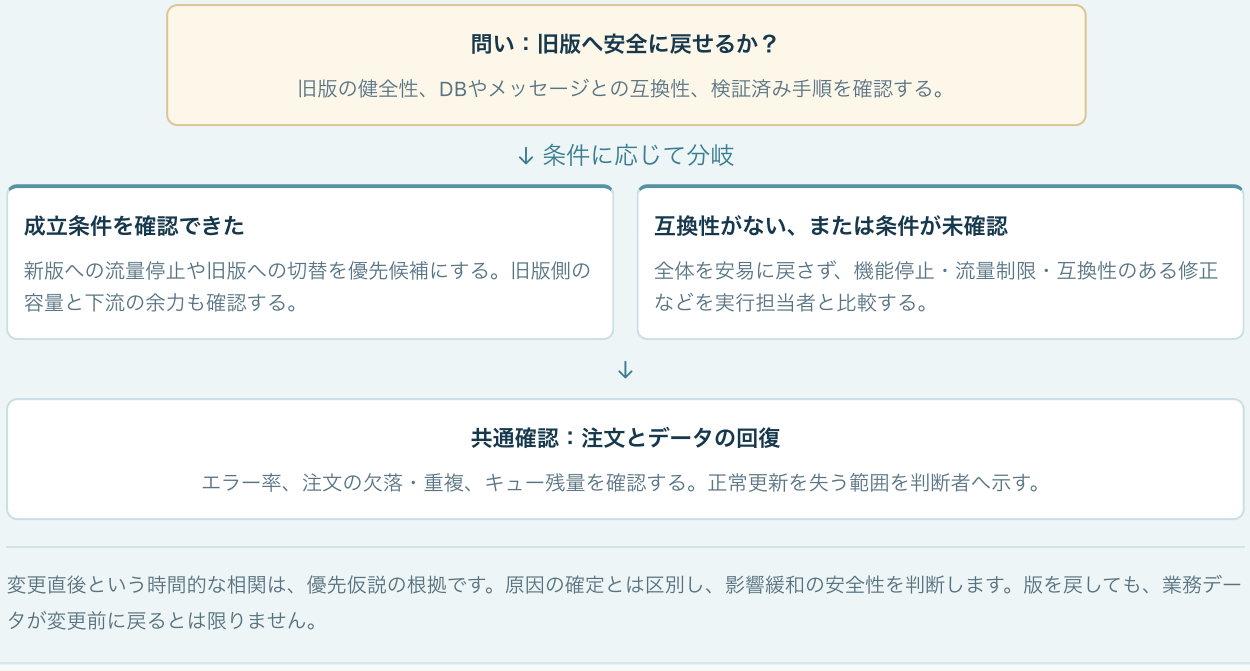
仮説と検証：結果によって次の行動を変える

1. 新バージョンの処理不具合：版ごとのエラー率とアプリ例外を比べます。新版だけで再現し旧版が健全なら、新版への流量を減らす案の根拠になります。

2. バックエンド選択・接続の問題：ヘルスチェック、配置、ポート、LBログを確認します。対象のLBではstatusDetailsでLB生成のエラーとバックエンド由来のエラーを区別できます。これはエラー発生箇所の手掛かりで、最終原因とは別です。
3. 共通依存先の飽和：新旧どちらでも同時に失敗するならDB接続や依存APIを確認します。版を戻しても下流の待ちが残る可能性があります。

影響を緩和する：副作用も一緒に考える

図 13　切り戻しは、戻せる条件を確認して選ぶ



変更直後という時間的な相関は、優先仮説の根拠です。原因の確定とは区別し、影響緩和の安全性を判断します。版を戻しても、業務データが変更前に戻るとは限りません。

後方互換性と手順を確認できれば、新版への配信停止や旧版への切り戻しを優先候補にします。互換性が不明なら、安易にデータを巻き戻さず、影響の少ない機能停止や流量制限を検討します。再試行は冪等性と負荷増幅を確認して制御します。

復旧の確認と予防

エラー率だけでなく、注文の成功・欠落・重複、キュー残量、正常版の容量を確認します。変更前と同じ負荷で安定しているかも見ます。

段階的リリース、監視に基づく停止条件、DB変更の互換性、切り戻し訓練を整えます。変更と障害の相関が強くても、事後検証で原因を確かめます。

TAMとして持つ役割

TAMは影響と対応方針を顧客へ説明し、開発・運用に変更差分、版別計測、戻せる条件を確認します。自分が実装担当であるとは仮定せず、判断者と実行者を明確にします。

口頭回答の例

> まず注文の失敗割合と、重複や結果不明の有無を確認します。リリース直後なので変更起因を優先仮説にしますが、新旧版のエラー率とLB・アプリのログで確かめます。旧版が健全でデータ互換性と手順を確認できれば、新版への流量を止めるなど安全な緩和を進めます。互換性が不明なら、単純な巻き戻しでデータを壊さないよう機能停止や流量制御も比較します。復旧は注文データの整合性まで確認し、段階的リリースと停止条件を改善します。

90〜150秒程度で説明するための練習用の例です。Google公式の回答時間制限ではなく、発話時間も実測していません。全文を暗記せず、判断を変える条件を自分の言葉で説明してください。

> 注意：「リリース直後だから原因確定」「5xxだからLB故障」「ロールバックは常に安全」の三つを避けます。

公式: External Application Load Balancerのトラブルシューティング

条件を変えた追加問題

新しい情報：新版はDBに新しい状態値を書き込み、旧版はその値を扱えません。旧版へ戻せばよいでしょうか。

解説 | 追加問題の考え方を開く

そのまま戻すと旧版がデータを誤処理する恐れがあります。書き込まれた範囲と互換性を開発・DB担当に確認し、書き込み機能の制限、互換性のある修正版、データ変換などを比較します。アプリのバイナリーと業務データは別の状態なので、デプロイを戻すだけで変更前に戻れるとは考えません。

ケース04｜サービスからだけアクセスが拒否される

誰が・何に・どの操作をしようとしたかを固定する。

> 学習用の創作ケースです。実際のGoogleの出題・採点基準を再現したものではありません。数値や構成は、このケースの前提です。

出題：分かっていることと、まだ分からないこと

開発者が手元で実行すると成功するファイル読込が、Google Cloud上のアプリからは権限エラーになります。アプリはサービスアカウントを使う想定です。実際に使ったIdentity、拒否された権限、リソース名は未確認です。

先に自分で考える

認証と認可をどう切り分け、最小権限のまま復旧させますか。

30〜60秒で考えを整理し、結論と理由を声に出してください。解説を読む前に、最初の質問と次の行動を決めます。

解説 | 解説を開く：判断の分岐と回答例

優先して確認する質問と、その理由

1. 「失敗した呼出しの実際のprincipal、対象リソース、必要なpermissionは何ですか」：想定の設定と実際の呼出しを分け、検証単位を固定します。
2. 「資格情報を取得できているか、どのエラーがどの時刻に出たか」：本人確認の失敗か、確認済みIdentityへの操作拒否かを区別します。
3. 「最近のIAM、実行環境、対象リソース、境界ポリシーの変更はありますか」：許可不足以外の制限やIdentityの取り違えも調べます。

仮説と検証：結果によって次の行動を変える

1. 別のIdentityで動作している：実行環境とログを確認します。想定外なら正しいIdentityを使う設定へ直し、人の広い権限をアプリへコピーしません。
2. 必要な許可がない、または条件を満たさない：対象の階層とポリシー、条件、拒否を確認します。対応範囲と調査権限を確かめた上でPolicy Troubleshooterも使えます。
3. IAM以外の境界で止まっている：IAM上の許可があっても拒否される場合、エラーに応じてサービス固有設定やネットワーク・データ境界などを調べます。IAMロールを増やし続ける対応はしません。

影響を緩和する：副作用も一緒に考える

必要な操作だけを対象リソースに許可するか、誤ったIdentity設定を修正します。緊急の一時権限が必要なら対象、期限、承認、監査、解除方法を決めます。Ownerなどの強い権限や長期鍵の配布を最初の解決策にしません。

復旧の確認と予防

実際のアプリのIdentityで目的の操作が成功することと、不要な操作は許可されていないことを確認します。人のアカウントでの成功は代わりの証拠になりません。

実行Identityの明示、権限差分のレビュー、環境別テスト、不要権限の棚卸しを行います。エラー共有ではtokenや鍵を貼らず、機密を除いた識別情報を使います。

TAMとして持つ役割

TAMはIdentity・対象・操作の三点と変更時刻を整理し、顧客のIAM管理者、アプリ担当、必要ならSupportをつなぎます。権限変更の判断者と期限を確認します。

口頭回答の例

> まず実際に呼び出したIdentityと対象、必要な操作を確認します。開発者の手元で成功しても、アプリと同じ権限とは限りません。資格情報の取得に失敗しているのか、認証後に操作を拒否されているのかを分けます。その上でポリシーや条件、拒否、対象範囲を確認し、必要ならPolicy Troubleshooterを使います。復旧は必要最小限の許可かIdentity設定の修正で行い、実際の実行Identityで成功と権限制限の両方を検証します。

90〜150秒程度で説明するための練習用の例です。Google公式の回答時間制限ではなく、発話時間も実測していません。全文を暗記せず、判断を変える条件を自分の言葉で説明してください。

> 注意：「権限エラー＝認証失敗」とまとめず、誰かを確認する段階と、何を許すかの段階を区別します。

公式: Policy Troubleshooterによるアクセス確認

条件を変えた追加問題

新しい情報：必要なロールは付いており、IAMの検証でも許可と見えます。しかしサービスは引き続き拒否します。

解説 | 追加問題の考え方を開く

ロール追加を続けず、まず検証したprincipal・resource・permissionと実際の失敗が一致しているかを確かめます。一致するなら、エラー詳細からIAM以外の境界やサービス固有条件へ進みます。許可ポリシー上の評価は、実際の呼出しに必要な全条件の成立を保証するものではありません。

ケース05｜キャンペーンで負荷が急増した

増強だけでなく、下流保護と業務上の優先順位を考える。

> 学習用の創作ケースです。実際のGoogleの出題・採点基準を再現したものではありません。数値や構成は、このケースの前提です。

出題：分かっていることと、まだ分からないこと

キャンペーン開始後、要求数が通常の5倍になり、応答遅延とタイムアウトが増えています。Webアプリは自動スケールしますがDBは単一の接続先です。注文と商品検索が同じ系統を使っています。

先に自分で考える

自動スケールしているのに遅い理由をどう考え、どの順で影響を抑えますか。

30〜60秒で考えを整理し、結論と理由を声に出してください。解説を読む前に、最初の質問と次の行動を決めます。

解説 | 解説を開く：判断の分岐と回答例

優先して確認する質問と、その理由

1. 「注文と検索のどちらにどの程度影響し、何を優先して守る必要がありますか」：流量制御や機能縮退の業務判断に使います。
2. 「アプリ数、同時実行数、DB接続、待ち行列、再試行はどう変化しましたか」：容量不足の場所と負荷増幅を探します。
3. 「キャンペーンの継続時間、予測負荷、利用可能な容量や制限は分かりますか」：一時緩和と追加容量の実現性を判断します。

仮説と検証：結果によって次の行動を変える

1. アプリの処理容量不足：実際のスケール数と立ち上がり、CPU以外のメモリーや同時処理の上限を確認します。増強で処理量が伸びるなら段階的に増やします。

2. DBや依存先が飽和：DB待ち、接続数、ロック、クエリー時間を確認します。アプリを増やしてもDB待ちが伸びるなら、接続制御と下流保護を優先します。
3. 再試行による増幅：元の要求数と試行回数を比べます。タイムアウトで同じ仕事が繰り返されていれば、回数制限、待ち時間、重複処理対策を検討します。

影響を緩和する：副作用も一緒に考える

注文など重要な処理の容量を確保し、負荷の高い補助機能の制限、キャッシュ可能な読込の利用、流量制限を比較します。キューへ移す案は、遅延を許容し、滞留上限と処理結果の通知を設計できる場合に使います。無制限のキューは障害を後ろへ延ばすだけです。

復旧の確認と予防

要求数に対する成功処理量、遅延、拒否数、重複、キュー滞留が業務上の条件を満たすか確認します。全要求を受け付けたことと、正しく処理できたことを分けます。

負荷試験は下流依存先を含めて行い、容量の頭打ち、スケールの時間差、縮退条件を確認します。キャンペーン前に運用・業務・Supportの連絡体制を決めます。

TAMとして持つ役割

TAMは重要業務と許容できる縮退を顧客と合意し、アプリ・DB・容量の担当をつなぎます。自動スケールだけで無制限に追従できるとは説明しません。

口頭回答の例

> まず注文など守るべき処理と、遅延や失敗の業務影響を確認します。自動スケールはアプリの容量を増やしても、DBや依存先の容量まで解決するとは限りません。実際のスケールとDB接続・待ち、再試行を見てボトルネックを絞ります。下流が飽和していれば、アプリ増強だけを続けず、流量制御や補助機能の縮退で重要処理を守ります。復旧は成功処理量と遅延、重複や滞留で確認し、次回は依存先を含む負荷試験へつなげます。

90〜150秒程度で説明するための練習用の例です。Google公式の回答時間制限ではなく、発話時間も実測していません。全文を暗記せず、判断を変える条件を自分の言葉で説明してください。

> 注意：「リトライすれば成功率が上がる」とだけ説明しません。過負荷では試行数の増加が飽和を悪化させることがあります。

公式: Google SRE — Handling Overload

条件を変えた追加問題

新しい情報：顧客は「429や受付制限は絶対に出したくない」と言っています。どう説明しますか。

解説 | 追加問題の考え方を開く

受付を制限しない場合の待ち時間、タイムアウト、全体停止のリスクを具体的に説明します。必要容量を実際に確保できるか確認し、受付待ちを許す業務、優先処理、キャンペーン流量の調整を比較します。制限の表示を避けることと、全利用者の処理を完了することは異なるため、顧客の成功条件を確認して判断します。

ケース06｜更新直後の表示が古く、検索も遅い

整合性と性能を別の要求として扱い、保存・読込経路をたどる。

> 学習用の創作ケースです。実際のGoogleの出題・採点基準を再現したものではありません。数値や構成は、このケースの前提です。

出題：分かっていることと、まだ分からないこと

ユーザーが住所を変更した直後、画面に古い住所が表示されることがあります。同じ時期に検索も遅くなりました。アプリはキャッシュとDBを使っています。読込先がレプリカか、更新がcommit済みかは不明です。

先に自分で考える

二つの症状をどう整理し、どの証拠で仮説を分けますか。

30〜60秒で考えを整理し、結論と理由を声に出してください。解説を読む前に、最初の質問と次の行動を決めます。

解説 | 解説を開く：判断の分岐と回答例

優先して確認する質問と、その理由

1. 「何を更新し、どの画面・利用者で、いつまで古い値が見えますか」：一時的な古さの許容条件と影響範囲を確認します。
2. 「更新は成功が確認され、読込はどのDBやキャッシュへ向かいますか」：画面表示だけで保存失敗とせず、書込・読込経路を区別します。
3. 「遅延は更新と検索のどちらにあり、直近にクエリー、データ量、接続先の変更はありますか」：性能の原因を整合性の問題と切り分けます。

仮説と検証：結果によって次の行動を変える

1. キャッシュの古さ：DBの確定値と画面の値を同時刻で比較します。DBは新しくキャッシュだけ古いなら、無効化や期限、キーの付け方を調べます。キャッシュ全削除はDB負荷を急増させるため対象を絞ります。
2. 読込先や反映時差：書込先と読込先、製品・構成の整合性特性を確認します。反映待ちが原因なら、更新直後の読込経路を変える案と負荷を比較します。すべてのレプリカが同じ遅延特性とは考えません。
3. クエリーや競合の性能問題：実行計画、読込量、ロック待ち、同時実行を確認します。待ちが大きければ競合、読込量が増えていれば条件や索引を調べます。CPUだけで判断しません。

影響を緩和する：副作用も一緒に考える

重要な更新の確認を正しい保存先で行い、影響対象のキャッシュ無効化や、整合性を満たす読込経路への切替を検討します。ただし負荷増加と接続余力を確認します。索引追加やクエリー変更も、書込負荷と実行影響を見て小さく試します。

復旧の確認と予防

更新した値が必要な時間内に表示されることと、検索の応答時間・成功率を別々に確認します。データの欠落や二重更新がないかも対象を絞って調べます。

読込・書込経路と整合性要件を設計時に明記し、更新直後のテストとデータ量増加時の負荷試験を追加します。性能と整合性のトレードオフを操作ごとに決めます。

TAMとして持つ役割

TAMは業務上の古さの許容、保存成功の証拠、経路、性能を整理します。アプリ担当にはキャッシュと接続先、DB担当には確定値・待機・実行計画を依頼し、両者を同じ時間軸で合わせます。

口頭回答の例

> まず古い住所が見える操作と継続時間、業務影響を確認します。保存失敗、キャッシュの古さ、読込先への反映待ちを分けるため、更新の確定と読込経路を確かめます。検索遅延は別に計測し、DB待ちや実行計画を見ます。キャッシュが原因なら対象を絞って修正し、整合性を満たす読込へ変える際はDB負荷も確認します。回復は値の正しさと応答性能の両方で確かめ、操作ごとの整合性要件を明確にします。

90〜150秒程度で説明するための練習用の例です。Google公式の回答時間制限ではなく、発話時間も実測していません。全文を暗記せず、判断を変える条件を自分の言葉で説明してください。

> 注意：レプリケーション、バックアップ、キャッシュは目的が異なります。「コピーがある」だけで最新性や復旧性を保証しません。

条件を変えた追加問題

新しい情報：古い値が見えるのは残高照会です。顧客は最新の確定残高が必要だと説明しました。判断は変わりますか。

解説 | 追加問題の考え方を開く

許容できる古さがほぼない重要操作なら、利便性のための古いキャッシュ表示を同じ条件で続ける案は採れません。どの時点の確定残高かを定義し、その要求を満たすデータアクセスを選びます。性能が厳しい場合も、正しさの条件を黙って緩めず、設計・容量・表示の仕方を顧客と比較します。

ケース07｜一つのZoneが使えなくなった

配置数だけでなく、状態・容量・切替経路まで確認する。

> 学習用の創作ケースです。実際のGoogleの出題・採点基準を再現したものではありません。数値や構成は、このケースの前提です。

出題：分かっていることと、まだ分からないこと

一つのZoneでアプリVMにアクセスできなくなりました。顧客は「VMを2台にしたので冗長化済み」と説明しています。2台の配置、DB、セッションの保存先、健全側の容量は未確認です。

先に自分で考える

サービス継続のために何を確認し、HA構成の不足をどう見つけますか。

30〜60秒で考えを整理し、結論と理由を声に出してください。解説を読む前に、最初の質問と次の行動を決めます。

解説 | 解説を開く：判断の分岐と回答例

優先して確認する質問と、その理由

1. 「どの業務が停止し、どの時間までにどの程度の処理を戻す必要がありますか」：回復目標と対応の優先順位を確認します。
2. 「2台は異なるZoneですか。DB・共有ストレージ・セッションなどの依存先はどこですか」：同一障害で失われる構成要素を探します。
3. 「健全側へ流せる経路と容量はあり、切替は検証済みですか」：台数だけでなくサービス継続の実現性を判断します。

仮説と検証：結果によって次の行動を変える

1. 配置上の単一障害点：2台が同じZoneなら同時に影響を受けます。別ZoneでもDBが片側だけなら、アプリの分散だけでは業務を継続できません。依存関係を図にします。

2. 切替経路や状態の問題：健全なバックエンドへ到達できるか、セッションやローカルファイルに依存していないか確認します。切替後にログインや書込が失敗するなら、その状態を調べます。
3. 健全側の容量不足：切替で要求が集中し、待ちやエラーが増えていないか確認します。容量が不足するなら追加確保と優先業務への配分を検討します。

影響を緩和する：副作用も一緒に考える

検証済みの健全側へ流量を寄せ、必要なら重要業務を優先します。DBなど状態を持つ系の昇格は、書込主体と整合性を確認し、二重書込を避けます。復旧見込みが不明なZoneの回復だけを待たず、可能な継続策を並行して調べます。

復旧の確認と予防

別Zoneからの業務操作、書込の一貫性、容量、エラー率を確認します。インスタンスが起動したことだけでサービス回復とは扱いません。

障害単位に合わせた配置、依存先の冗長性、障害時容量、切替訓練を整えます。HAは障害影響を抑える設計であり、あらゆるデータ損失やRegion障害を自動的に解決するものではありません。

TAMとして持つ役割

TAMは障害範囲、業務影響、利用可能な継続策を整理し、クラウド側調査と顧客側の切替判断をつなぎます。公式障害情報と、この顧客で確認できた事実を区別して伝えます。

口頭回答の例

> まず影響業務と回復目標を確認し、2台のVMと依存先がどの障害単位に分散されているかを見ます。別Zoneにアプリがあっても、DBやセッションが失われれば業務は継続できません。健全側への経路と容量、状態の扱いを確認し、検証済みの切替を実行担当者と進めます。書込系は二重書込とデータ整合性に注意します。回復は実際の業務処理と容量で確認し、障害単位ごとの構成と切替訓練を改善します。

90〜150秒程度で説明するための練習用の例です。Google公式の回答時間制限ではなく、発話時間も実測していません。全文を暗記せず、判断を変える条件を自分の言葉で説明してください。

> 注意：台数を増やすこと、障害単位を分けること、切替後に業務を続けられることを分けて確認します。

条件を変えた追加問題

新しい情報：異なるZoneにアプリを置いていましたが、セッションを各VMのメモリーだけに保存していました。

解説 | 追加問題の考え方を開く

切替でログイン状態や途中処理が失われる可能性を説明します。再認証で安全に再開できるか、途中注文の状態は別の永続先にあるかを確認します。将来は共有または外部の状態管理を比較しますが、その保存先自体の可用性と負荷も設計します。「ステートレス化」と言うだけでは状態は消えず、どこで管理するかの判断が必要です。

ケース08｜バックアップは成功しているが、復旧できるか

バックアップ取得と、業務の復旧時間・復旧地点を分ける。

> 学習用の創作ケースです。実際のGoogleの出題・採点基準を再現したものではありません。数値や構成は、このケースの前提です。

出題：分かっていることと、まだ分からないこと

顧客は毎晩バックアップを取得しています。誤更新が発生し、午前10時に発覚しました。顧客は「30分以内に復旧し、失ってよいデータは15分分まで」と説明しています。復元試験の実績と、ログ等による時点復旧の可否は不明です。

先に自分で考える

現在の証拠で目標を満たせると言えますか。何を確認し、どう対応を組み立てますか。

30〜60秒で考えを整理し、結論と理由を声に出してください。解説を読む前に、最初の質問と次の行動を決めます。

解説 | 解説を開く：判断の分岐と回答例

優先して確認する質問と、その理由

1. 「誤更新の開始時刻・対象と、正常データをどこまで失ってよいかは確認されていますか」：戻す地点とデータの扱いを決めます。発覚時刻と発生時刻を分けます。
2. 「使えるバックアップとログ、復元に必要な鍵・権限・容量はありますか」：復元可能な地点と依存条件を確認します。
3. 「過去の復元所要時間と、アプリ接続・整合性確認まで含む手順はありますか」：RTOの実現性を評価します。取得成功だけでは所要時間が分かりません。

仮説と検証：結果によって次の行動を変える

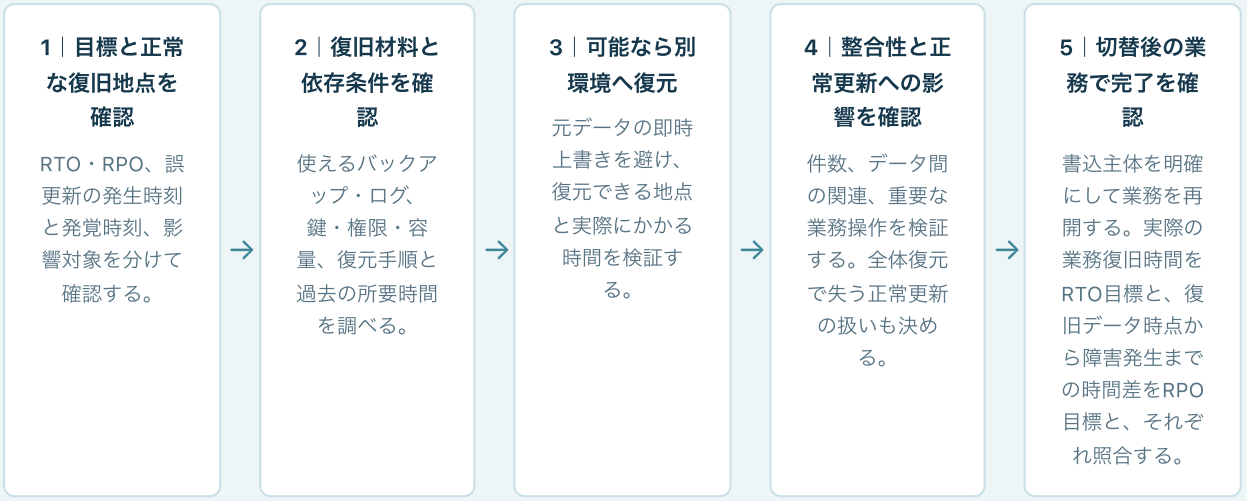
1. 目標に合う復旧地点がない：夜間のバックアップしかなければ、15分のRPOを満たせない可能性があります。実際に利用可能な復旧地点と必要な正常更新を確認します。
2. 復旧地点はあるが手順が長い：データ量、復元先確保、鍵や依存先、切替・検証時間を測ります。30分を超えるなら、その差を隠さず代替の業務継続策を比較します。
3. 全体復元で正常更新も失う：影響範囲を絞り、別環境へ復元して差分を検証できるか確認します。整合性のある選択的修復が可能かはDB・アプリ担当が判断します。

影響を緩和する：副作用も一緒に考える

誤更新が継続中なら書込制限などで被害拡大を抑え、証拠と復旧材料を保全します。復元は元データをすぐ上書きするのではなく、可能なら隔離した復元先で検証します。正常な業務更新を失う範囲と、業務停止の影響を顧客の判断者へ示します。

復旧の確認と予防

図 14　**バックアップ取得から、業務の復旧まで**



「取得成功」と「目標内に正しく復旧できる」は別です。誤更新の継続を止める判断と証拠・復旧材料の保全は並行します。最新の保存時点にも誤更新が含まれる可能性があります。

データの件数・関連整合性・重要な業務操作を確認し、切替後の書込主体を明確にします。30分のRTOは復元処理単体ではなく、合意した業務回復の終点までで評価します。

復元試験で実測し、必要なら取得間隔や時点復旧、保管分離、復元先準備を見直します。RPOを短くする案は保存と運用の費用も含めて比較します。

TAMとして持つ役割

TAMは業務目標、確認済みの復旧能力、差分、選択肢を整理します。データ復元の専門担当に実行と検証を依頼し、未検証のRTOや復旧地点を保証しません。

口頭回答の例

> 夜間バックアップの成功だけでは、30分のRTOと15分のRPOを満たすとは言えません。誤更新の時刻と対象、利用可能なバックアップ・ログ、鍵や容量、復元試験の実績を確認します。同時に誤更新の継続を止め、復旧材料を保全します。全体復元で正常更新を失う可能性もあるため、別環境への復元と差分確認を比較します。回復はデータ整合性と業務操作まで確認し、目標との差があれば顧客に明示して業務継続策を決めます。

90〜150秒程度で説明するための練習用の例です。Google公式の回答時間制限ではなく、発話時間も実測していません。全文を暗記せず、判断を変える条件を自分の言葉で説明してください。

> 注意：RTOとRPOは目標です。バックアップ頻度だけから、実現済みの復旧能力として説明しません。

公式: Google Cloud — Disaster recovery planning guide

条件を変えた追加問題

新しい情報：午前10時に発覚しましたが、誤更新は午前8時から始まっていました。9時55分の状態へ戻せばRPOを満たしますか。

解説 | 追加問題の考え方を開く

9時55分の状態にも誤更新が含まれる可能性があります。RPOの時間幅だけで選ばず、どの地点が正常か、8時以降の正常更新をどう再適用するかを確認します。障害・誤更新の性質によって、データの正しさを回復できる地点と最新の保存地点は一致しません。

ケース09｜月の途中でクラウド費用が急増した

利用量・単価・構成・業務価値を分けて調べる。

> 学習用の創作ケースです。実際のGoogleの出題・採点基準を再現したものではありません。数値や構成は、このケースの前提です。

出題：分かっていることと、まだ分からないこと

顧客が「今月のクラウド費用が急に増えた」と相談しています。新機能を公開した週と重なりますが、業務量が増えたか、請求上の変化かは不明です。顧客は即座に費用を下げたいと言っています。

先に自分で考える

何を比べ、どの対応を優先しますか。安くするための停止には何を確認しますか。

30〜60秒で考えを整理し、結論と理由を声に出してください。解説を読む前に、最初の質問と次の行動を決めます。

解説 | 解説を開く：判断の分岐と回答例

優先して確認する質問と、その理由

1. 「確定請求か予測・見積額か、どの期間と比較していますか」：比較の意味と未確定性をそろえます。日数や表示条件も確認します。
2. 「増分はどのproject・service・SKU・regionに集中していますか」：合計額を分解し、調査対象を特定します。
3. 「要求数や保存量などの業務量と、費用・構成・契約上の条件はどう変わりましたか」：利用増と単価・効率の変化を分けます。

仮説と検証：結果によって次の行動を変える

1. 正当な利用増：要求数やデータ量と費用を比べます。総額が増えても一処理当たりの費用が改善しているなら、予算と事業価値の判断になります。

2. 非効率な構成や処理：重複実行、不要な保持、過剰な容量、データ転送などの増分を確認します。新機能だけで増えたなら、処理経路や保存・転送の変化を調べます。
3. 想定外の利用や請求条件：不審なリソース作成や契約・表示条件の変化を確認します。侵害が疑われるなら費用最適化だけでなくセキュリティ対応へ接続します。

影響を緩和する：副作用も一緒に考える

所有者が確認できた不要リソースや重複処理から停止・縮小を検討します。本番停止、保持期間短縮、ログ削減は業務、復旧、監査の要件を確認してから行います。長期の契約的な割引は、利用量が不確かな初動の削減策として急いで選びません。

復旧の確認と予防

同じ比較条件で増分が止まったかを確認し、業務処理量、遅延、可用性が悪化していないかも測ります。請求情報の更新には時間差があり得るため、リソース側の実態と合わせます。

費用の所有者、用途ラベル、予算と通知、処理当たり費用を整えます。Cloud Billingの異常検知は調査開始の手掛かりとして使い、通知だけで自動的に支出を止める仕組みとは説明しません。

TAMとして持つ役割

TAMは増分の内訳と事業影響を整理し、顧客の運用・開発・財務担当で削減案の条件を合わせます。削減見込みは検証前の推定と、実際に確認した額を分けます。

口頭回答の例

> まず予測か確定額か、比較期間と表示条件をそろえ、費用増分をproject・service・SKUなどに分解します。次に業務量と単位当たり費用、構成や処理の変化を比較します。不要リソースや重複処理を優先して調べますが、本番停止や保持削減は業務・復旧・監査への影響を確認します。削減後は費用だけでなく性能と信頼性を測り、所有者と予算通知、単位当たり費用を運用に組み込みます。

90〜150秒程度で説明するための練習用の例です。Google公式の回答時間制限ではなく、発話時間も実測していません。全文を暗記せず、判断を変える条件を自分の言葉で説明してください。

> 注意：利用を止めれば必ずよいとは限りません。コスト削減による停止、データ損失、監査証跡不足も比較対象にします。

公式: Cloud Billing — コスト異常の確認と管理

条件を変えた追加問題

新しい情報：要求数は3倍、費用は2倍になっていました。費用が増えたことを問題と断定できますか。

解説 | 追加問題の考え方を開く

単位当たり費用は元の約3分の2です。効率は改善している可能性がありますが、予算超過、利益率、今後の需要を別に確認します。総額だけで無駄とせず、顧客が守るべき予算と事業価値の両方で判断します。

ケース10｜短期間でGoogle Cloudへ移行したい

移行方式を決める前に、目的・依存関係・戻せる条件を確認する。

> 学習用の創作ケースです。実際のGoogleの出題・採点基準を再現したものではありません。数値や構成は、このケースの前提です。

出題：分かっていることと、まだ分からないこと

顧客は既存クラウドからGoogle Cloudへ業務システムを移行したいと相談しています。期限は厳しく、停止可能時間は短いとのことです。アプリはVMとDBで動いており、外部接続、ライセンス、データ量は未確認です。

先に自分で考える

最初からサービスの置換表を示す以外に、何を確認し、どのように段階を分けますか。

30〜60秒で考えを整理し、結論と理由を声に出してください。解説を読む前に、最初の質問と次の行動を決めます。

解説 | 解説を開く：判断の分岐と回答例

優先して確認する質問と、その理由

1. 「移行の目的と成功条件、期限の理由、停止とデータ損失の許容は何ですか」：速度、費用、機能改善のどれを優先するか判断します。
2. 「アプリ、DB、Identity、ネットワーク、外部接続、バッチの依存関係は何ですか」：一部だけ移すことの成立条件を確認します。
3. 「互換性・ライセンス・データ量と、切替失敗時に戻す方法は確認されていますか」：候補方式の実現性とリスクを見ます。

仮説と検証：結果によって次の行動を変える

1. 変更を抑えた移行が適する：期限が厳しく互換性があるなら、実行環境の変更を限定する案を比較します。ただし運用負荷や将来の改善は別途残ると説明します。
2. 一部のマネージド化が有効：運用上の課題を解消できるなら候補にしますが、API・DB機能・接続・Identityの差を検証します。名称が似たサービス同士でも同等機能とは限りません。
3. 一括切替が危険：依存が多い、データが大きい、戻しが難しい場合は、検証環境、限定業務、段階的切替を比較します。移行中の二重運用と整合性の管理が増える点も評価します。

影響を緩和する：副作用も一緒に考える

検証で代表操作、性能、権限、監視、バックアップを確認してからリハーサルを行います。切替前にGo/No-Goの条件、担当、判断時刻、戻す条件を決めます。新環境へ書き込んだ後に旧環境へ戻す場合は、差分データをどう扱うかまで必要です。

復旧の確認と予防

機能の成功だけでなく、データ整合性、業務処理量、性能、運用手順、費用を成功条件と照合します。移行完了後も旧環境をいつ廃止できるか、保持要件と戻し期間を確認します。

依存関係とテスト結果を更新し、移行後に残した運用課題を別の改善項目として管理します。期限内の移行と最終的な最適化を、測れる成果で段階分けします。

TAMとして持つ役割

TAMは目的・成功条件・リスクを整理し、顧客の業務責任者、アプリ・DB・ネットワーク担当、必要な移行の専門チームをつなぎます。自分が移行設計や実行を一人で担うとは説明しません。

口頭回答の例

> まず移行の目的、期限、停止とデータ損失の許容を確認します。次にアプリから外部接続まで依存関係と互換性を整理します。期限を優先するなら変更を限定する案、運用改善を重視するなら一部のマネージド化を比較し、代表業務で検証します。切替はリハーサルとGo/No-Go、ロールバック条件を整え、新環境への書込後のデータも考慮します。完了は動作だけでなく、性能・整合性・運用・費用で確認します。

90〜150秒程度で説明するための練習用の例です。Google公式の回答時間制限ではなく、発話時間も実測していません。全文を暗記せず、判断を変える条件を自分の言葉で説明してください。

> 注意：最初から「Cloud Runへ移す」「DBをこの製品にする」と決めず、制約と依存関係から候補を選びます。

条件を変えた追加問題

新しい情報：顧客は「切替後に問題があればDNSを戻せばよい」と言っています。何が不足していますか。

解説 | 追加問題の考え方を開く

DNS変更の反映やキャッシュだけでなく、新旧環境へ書込が分かれる可能性を確認します。旧環境を再開する前に、新環境で受け付けた注文などをどう扱うか、書込主体をどう一つにするかを決める必要があります。宛先を戻す操作は、業務データを変更前へ戻すこととは別です。

ケース11｜不審なアクセスと費用増加が見つかった

侵害を断定せず、封じ込め・証拠・連絡の優先順位を考える。

> 学習用の創作ケースです。実際のGoogleの出題・採点基準を再現したものではありません。数値や構成は、このケースの前提です。

出題：分かっていることと、まだ分からないこと

運用担当者が、想定していないサービスアカウントの操作と、新しいリソースの作成を見つけました。費用も増えています。正当な自動処理か、資格情報の不正利用かは未確認です。業務への影響とデータ持出しの有無も不明です。

先に自分で考える

通常障害の調査と何が異なりますか。最初の対応と、避けるべき断言を説明してください。

30〜60秒で考えを整理し、結論と理由を声に出してください。解説を読む前に、最初の質問と次の行動を決めます。

解説 | 解説を開く：判断の分岐と回答例

優先して確認する質問と、その理由

1. 「対象のIdentity・リソース・時刻・操作と、その根拠は何ですか」：申告とログ上の事実を分け、対象範囲を固定します。
2. 「正当な変更や自動処理と一致しますか。現在も操作は続いていますか」：誤検知の可能性と、封じ込めを急ぐ必要性を確認します。
3. 「顧客のセキュリティ対応責任者と、業務・対外説明の判断者は誰ですか」：適切な体制を直ちに呼び、権限と連絡経路を合わせます。

仮説と検証：結果によって次の行動を変える

1. 正当な処理の変更：変更管理と実行主体を照合します。一致しても不要な権限や予算逸脱がないかを確認し、根拠を記録します。
2. 資格情報の不正利用：監査記録、認証・権限変更、アクセス先を確認します。関連するIdentityやリソースへ調査を広げますが、見えない範囲を「影響なし」とは扱いません。
3. 侵害された実行環境：不審なプロセスや通信、作成経路などを専門担当が調べます。リソースを削除する前に、証拠保全と継続被害のリスクを比較します。

影響を緩和する：副作用も一緒に考える

セキュリティ責任者の指揮で、必要なIdentityの無効化、権限制限、ネットワーク隔離などを検討します。即時削除は証拠を失うため慎重に判断しますが、保全完了まで被害を無制限に放置もしません。封じ込めで止まる業務と代替手段も確認します。

復旧の確認と予防

不審な動作が止まっただけで完全解決とせず、侵入経路、影響範囲、残る資格情報や永続化の可能性を専門担当が確認します。SCCを利用している場合はfindingの詳細と関連リスクを調査材料にできます。

最小権限、長期鍵の扱い、Identityの分離、監査と検知、対応訓練を見直します。対外報告や法的な判断は、顧客の所定の責任者へつなぎます。

TAMとして持つ役割

TAMは事実、未確認、実施済み措置、担当、連絡時点を整理し、顧客のセキュリティ担当と必要なSupport等を連携させます。「漏えいなし」「完全に安全」と未確認の結論を出しません。

口頭回答の例

> まず不審と判断した操作、Identity、対象、時刻の証拠を確認し、顧客のセキュリティ対応責任者を呼びます。正当な処理の可能性と継続中の不正利用を分けて調べます。被害が続く恐れがあれば、証拠保全と並行して権限制限や隔離を判断しますが、業務停止の影響も共有します。不審な動作が止まっただけでは完了にせず、専門担当と侵入経路と影響範囲を確認します。顧客には確認済みの事実と未確認事項を分けて伝えます。

90〜150秒程度で説明するための練習用の例です。Google公式の回答時間制限ではなく、発話時間も実測していません。全文を暗記せず、判断を変える条件を自分の言葉で説明してください。

> 注意：検知は調査の入口です。findingの存在を侵害確定とせず、findingがないことを安全の証明にもしません。

公式: Security Command Center — 脅威の調査と対応

条件を変えた追加問題

新しい情報：顧客から「攻撃者が作ったVMを今すぐ削除してよいか」と聞かれました。

解説 | 追加問題の考え方を開く

継続被害を止める緊急性、ネットワーク隔離などの代替、必要な証拠の取得をセキュリティ責任者と判断します。削除すれば調査材料を失う可能性があります。一方で保全のために危険な通信を継続させるのも不適切です。所定の権限と手順で封じ込めを先行し、保全と削除の順序を決めます。

ケース12｜アラートが多く、手作業が減らない

顧客の成功を測る指標から、通知と改善投資を組み直す。

> 学習用の創作ケースです。実際のGoogleの出題・採点基準を再現したものではありません。数値や構成は、このケースの前提です。

出題：分かっていることと、まだ分からないこと

運用チームはCPUアラートへの対応と、毎日の再起動・手動集計に追われています。顧客は「障害は少ないが運用が苦しい」と説明しています。利用者の失敗率、対応工数、アラートの有効性は測っていません。

先に自分で考える

SLI/SLOとToil削減を使って、どこから改善しますか。自動化の前に何を決めますか。

30〜60秒で考えを整理し、結論と理由を声に出してください。解説を読む前に、最初の質問と次の行動を決めます。

解説 | 解説を開く：判断の分岐と回答例

優先して確認する質問と、その理由

1. 「利用者が成功したと判断する操作と、困る遅延・失敗は何ですか」：技術リソースの状態から、顧客が必要なサービスの品質へつなぎます。
2. 「通知のうち実際に行動が必要だった割合と、対応時間は分かりますか」：ノイズと必要な通知を分ける材料を集めます。
3. 「手作業の頻度・工数・失敗時の影響と、毎回同じ判断で済む部分は何ですか」：削減効果と自動化リスクを比較します。

仮説と検証：結果によって次の行動を変える

1. 利用者影響と通知がずれている：CPU上昇の通知を、要求成功率や遅延と照合します。CPUは容量調査に使いつつ、夜間に人を起こす条件は行動可能な影響へ合わせます。

2. 繰り返す手作業が対症療法：再起動が必要な原因、効果、失敗例を確認します。自動再起動だけで問題を隠さず、原因対策と安全な自動化を比較します。
3. 優先順位を決める基準がない：対象操作、計測場所、良い要求と有効な要求の定義、期間、目標を決めます。SLOとerror budgetを、信頼性改善と変更の判断材料として使います。

影響を緩和する：副作用も一緒に考える

最初は一つの重要な利用者操作と、一つの負担が大きい手作業に絞ります。通知を止める場合は代わりの検知と見逃し確認を用意します。自動化は小さな対象から始め、権限、停止条件、失敗通知、手動復帰を整えます。

復旧の確認と予防

利用者の成功率・遅延を悪化させず、対応工数と不要な呼出しが減ったかを測ります。自動化の成功件数だけでなく、誤動作や保守工数を含めます。

SLOは一度で完全に決めようとせず、顧客・開発・運用で合意し実測から見直します。月次で工数と再発を確認し、削減した時間をどの改善へ使うか決めます。

TAMとして持つ役割

TAMは業務の成功条件を技術指標へ落とす議論を進め、関係者が同じ判断材料を持てるようにします。運用負荷を個人の努力で吸収させず、効果とリスクのある改善提案へ変えます。

口頭回答の例

> まず重要な利用者操作と、許容できる遅延・失敗を確認します。CPUアラートを消すこと自体を目的にせず、利用者影響と通知後の行動を見ます。並行して手作業の頻度、工数、リスクを測り、一つの高効果な対象から改善します。再起動などの対症療法は原因も調べ、自動化には権限・停止条件・失敗通知を設けます。成果は利用者品質を維持した上で、不要な通知と運用工数が減ったかで確認します。

90〜150秒程度で説明するための練習用の例です。Google公式の回答時間制限ではなく、発話時間も実測していません。全文を暗記せず、判断を変える条件を自分の言葉で説明してください。

> 注意：error budgetは「使い切ってよい障害枠」とだけ説明せず、利用者影響を踏まえて変更と信頼性投資を調整するための判断材料として扱います。

公式: Google SRE Workbook — Implementing SLOs

条件を変えた追加問題

新しい情報：100万件の有効な要求に対して、SLOは成功率99.9%です。失敗は800件でした。予算はどれだけ残っていますか。

解説 | 追加問題の考え方を開く

要求数ベースで、この期間と要求集合を使う前提なら許容失敗は100万×0.1%=1,000件です。800件失敗したので、残りは200件、予算の20%です。これは許容停止時間へそのまま変換できません。今後の変化と失敗の増え方、顧客影響、合意したerror budget policyを合わせて変更の判断をします。

直前に戻る、判断の早見

原因を早く言うより、根拠のある次の一手を言う。知識の説明も、条件と限界まで短くまとめる。

ケース回答の出発点

「まず対象業務と影響、発生範囲、時間制約を確認します。重大な影響があれば、関係者との連携と安全な緩和を並行します。現在の情報では〇〇を仮説に置き、△△で確かめます。□□が観測されれば案A、そうでなければ案Bを検討します。実行前に副作用と戻し方を確認し、顧客の業務と指標の両方で回復を確かめます。」

丸暗記して全文を読む必要はありません。与えられた情報に合わせ、必要な箇所だけ使います。用語説明では「定義→仕組み→使う条件→注意」を短く話せば十分な場合があります。

症状から最初の分岐へ

相談	最初に分ける	優先する証拠	早合点しやすい点
遅い	全体／一部、成功／失敗、どの測定区間か	利用者側時間、分布、トレース、待ちと負荷	CPUだけで原因を決める。
つながらない	DNS／接続／TLS／HTTP／アプリ	エラー、時刻、送信元・宛先、到達した層	Firewallを広く開ける。
権限エラー	誰が／どの操作を／何へ行ったか	実行主体、拒否された権限、ポリシー、監査記録	人の権限とサービスの権限を混同。
負荷増	CPU処理／待ち／依存先／上限	到着量、処理量、キュー、接続、飽和	台数増でDBへの負荷も増える。
復旧したい	インフラ障害／誤操作／破損	守るデータ、復旧点、復旧時間、依存先	レプリカがバックアップになると思う。
費用が増えた	利用量／単価／構成・契約／請求範囲	サービス・SKU・期間・地域別の内訳	すぐ本番容量を減らす。

比較するときの六つの軸

性能：必要な応答時間・処理量を満たすか。**信頼性**：どの故障に耐え、どの程度で戻れるか。**Security**：アクセス、露出、データ、監査を守れるか。**費用**：通常・繁忙・障害時の利用量と転送を含める。**運用**：変更、監視、復旧を担当できるか。**移行**：段階導入、データ同期、戻し方が成立するか。

短く言い直すときの修正例

- 「必ず復旧します」→「この条件なら緩和が見込めます。〇〇で効果を確認し、悪化時はΔΔします」。
- 「ログを全部見ます」→「対象時刻の失敗要求と正常要求を比べ、〇〇の仮説を確認します」。
- 「マルチリージョンにします」→「単一リージョンでは満たせない要件と、整合性・費用・運用の条件を確認します」。
- 「原因不明なので待ちます」→「原因は未確定ですが、影響を把握し、安全な緩和と調査を並行します」。
- 「経験がないので分かりません」→「その製品固有の挙動は確認が必要です。原理として〇〇を確認し、公式仕様と専門担当の確認で具体化します」。

自己評価は、できた行動で付ける

状態	判断の目安
未着手	まだ学んでいない。弱いと断定しない。
読んだ	説明を理解したが、再現を確認していない。
補助あり	ヒントや教材を見れば説明・適用できる。
自力で適用	教材を閉じ、理由と条件を説明できた。後日、条件を変えて再確認する。

11/10を見据えた、学習と復習の回し方

期間は10/31〜11/9の学習案。日程と使える時間が変われば、章数より再現できる論点を優先する。

約10日で、三周する

以下は最短11/10に備えるための提案です。面接日確定や一日の学習時間は確認していません。初回は全体の位置をつかみ、二周目で理由を説明し、三周目で条件の違うケースに使います。すべての節を同じ密度で読む必要はありません。

期間の目安	学ぶ対象	終えるときの確認
10/31〜11/1	全体マップ、共通手順、NetworkとCompute	一つの要求の経路と、遅延／接続不能の最初の分岐を説明する。
11/2〜11/3	Data、IAM／Security、観測	認証と認可、レプリカとバックアップ、Metrics／Logs／Tracesの違いを例で説明する。
11/4〜11/5	性能、信頼性・SRE、設計判断	SLOの計算、RTO／RPO、スケールの副作用、比較案の採否条件を説明する。
11/6〜11/7	ケースを領域横断で練習	解説を見る前に質問・仮説・対策・検証を話し、追加条件で判断を修正する。
11/8〜11/9	苦手項目、nozの経験、短い模擬面接	過去経験と仮想ケースを切り替える。復旧後の予防策と自分の役割を説明する。

一回の学習を短く閉じる

1. 読む前に、その節の問いへ一度答える。分からなければ、そのまま解説を読む。
2. 解説を閉じ、仕組みと判断理由を自分の言葉で説明する。
3. 一つの条件を変える。負荷が増えた、変更を戻せない、データ損失不可などで案を見直す。
4. 説明できなかった一点を記録し、対応する節へ戻る。
5. 後の回で別の順序・別のケースとして再確認する。

時間が短い日は、早見の一項目を説明し、ケースの追加質問に一つ答えるだけでも区切れます。本文を読むことだけで時間を使い切らず、見ずに説明する時間を残します。

元のチャットへ戻るときの依頼例

> 「この教材のNetworkingと観測を読みました。一次RRKの学習モードで、接続不能の小さな創作ケースを一つ出してください。最初はヒント・解説を出さず、私の回答を待ってください。回答後は、根拠と条件を確認し、直す一点を教えてください。」

模擬面接を行う場合も、練習設定と今回の採用案内を分けます。本書を読んだ状態からは、まず短い説明問題とケースを混ぜて理解を確かめ、45分の練習を希望するときに元チャットで依頼すると進めやすくなります。

直前に新しい知識を増やし過ぎない

残り時間が少ないときは、主要な原理、判断の理由、nozの確認済み経験へ戻ります。細かな制限値を無理に覚えるより、何を確認すれば判断が変わるかを明確にします。不明な仕様に出会ったら、原理として分かる範囲と、公式文書で確かめる範囲を説明する練習に使います。

復習　CHAPTER 30

出典と作成・確認の範囲

公式資料と、個人の学習記録・創作ケースを区別する。

章内の公式リンクが、説明の対象を確認する入口です。確認日：2026/10/31。サービスの最新仕様は、対象構成・操作に合うページで再確認します。

1. Google Cloud Well-Architected Framework
2. Google SRE — Effective Troubleshooting
3. Incident Response
4. VPC networks
5. Cloud DNS overview
6. Cloud Load Balancing overview
7. Retry strategy
8. Choosing a Network Connectivity product
9. Private access options for services
10. Compute Engine overview
11. GKE overview
12. Container runtime contract
13. About instance autoscaling
14. Maximum concurrent requests
15. Choose a disk type
16. Filestore overview
17. Cloud Storage overview
18. About high availability
19. Cloud SQL backups overview
20. Google Cloud databases
21. Spanner overview
22. Firestore overview
23. Bigtable overview
24. BigQuery overview
25. IAM overview
26. About resource hierarchy
27. Service accounts overview

28. Secret Manager best practices
29. Cloud Audit Logs overview
30. Monitoring Distributed Systems
31. Cloud Trace overview
32. Structured logging
33. Percentiles and distribution-valued metrics
34. Performance optimization pillar
35. Continuously monitor and improve performance
36. Google SRE — Handling Overload
37. Addressing Cascading Failures
38. Google SRE Workbook — Implementing SLOs
39. Google Cloud — Disaster recovery planning guide
40. Canarying Releases
41. Postmortem Culture
42. Eliminating Toil
43. Reliability pillar
44. Cost optimization pillar
45. Create, edit, or delete budgets and budget alerts
46. About migration planning
47. External Application Load Balancerのトラブルシューティング
48. Policy Troubleshooterによるアクセス確認
49. Cloud Billing — コスト異常の確認と管理
50. Security Command Center — 脅威の調査と対応

個人向けの調整に使った資料

「Google Cloud TAM技術面接を学ぶ」チャットの開始条件と直近のやり取り、Google Cloud TAMカジュアル面談記録・一次面接準備（2026/10/29）、Google Cloud TAM技術面接 学習メモ（2026/10/29）、Interview Story・Answer BankのBaseDB移行支援項目（本人確認2026/10/26）を参照しました。個別の顧客名、SR番号、社内URL、連絡先、顧客ログ原文は掲載していません。学習用の例・判断手順・表現例は教材として作成したものです。

検証の範囲

技術内容は一次資料と照合し、計算例と本文の整合性を確認します。HTMLの目次・検索・解答開閉・学習記録・画面幅への対応、PDFの文字とレイアウトは別に検証します。Google Cloud環境での構築・障害再現、採用担当者による内容の承認、noz自身の習熟度や合格可能性の測定を行ったものではありません。